

Het integreren van Python modules in een .NET rekenmodel.

Een proof-of-concept bij ILVO.

Roetyncck Caradoc.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. J. Willem

Co-promotor: Dhr. S. Bosch

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Tijdens een gesprek met ILVO kwam de vraag naar voren of het mogelijk was om een probleem op te lossen dat zij hadden. Het onderwerp kwam op omdat hun onderzoekers hun berekeningsmodellen willen testen en aanpassen, maar ze hebben geen ervaring genoeg met C# om dit zelf te doen. De onderzoekers hebben wel ervaring met Python en daarom leek het een goed idee om een Jupyter-notebook te maken waarin de onderzoekers kunnen experimenteren met de modellen.

Ik heb gebruik gemaakt van genAI voor de grammatica & zinsbouw te verbeteren. Veel dank aan mijn moeder voor het helpen bij het schrijven van deze bachelorproef, aan mijn copromoter, Samuel Bosch, en aan mijn promotor, Jan Willem, voor hun begeleiding en feedback tijdens het proces.

Samenvatting

Bij ILVO, het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek, draaien veel onderzoeksprojecten rond complexe rekenmodellen. Deze modellen beschrijven hoe je emissies moet berekenen, of hoe bepaalde bedrijfsparameters zich verhouden tot een bepaald resultaat. Het probleem: onderzoekers schrijven deze modellen op, maar IT-developers implementeren ze vervolgens in C# en .NET. Zodra dat gebeurd is, zijn onderzoekers min of meer afhankelijk van de IT-afdeling voor elke kleine aanpassing of test. De code is als een zwarte doos. Ze zien hun formule niet terug, ze begrijpen niet precies wat er ondertussen gebeurt, en elke keer dat ze iets willen proberen, moet de IT-afdeling er aan te pas komen.

Deze bachelorproef bestudeert hoe die kloof kan worden gedicht. Het gaat uit van documentatie bij ILVO, analyseert waar de wrijving zit, en beschouwt welke bestaande werkwijzen uit soortgelijke domeinen (literate programming, Jupyter notebooks, documentatietools) daar nuttig voor kunnen zijn. Met die inzichten is een proof-of-concept gebouwd als proof-point: kunnen onderzoekers meer autonomie krijgen en meer inzicht in hoe hun model werkt, zonder dat de IT-kant eronder lijdt?

Het blijkt inderdaad mogelijk. Door beter inzicht in de scheiding tussen wat onderzoekers moeten begrijpen en wat de implementatie doet, kan zo'n samenwerking veel vloeiender worden ingericht. Er is niet alles opnieuw nodig, maar wel betere documentatie, betere structuur, en ruimte voor onderzoekers om voorzichtig te experimenteren.

Inhoudsopgave

Lijst van figuren	ix
Lijst van tabellen	x
Lijst van codefragmenten	xi
1 Inleiding	1
2 Stand van zaken	4
2.1 Pythonnet	4
2.2 IronPython	6
2.3 Literate programming	6
2.4 Org-mode Babel	6
2.5 Jupyter-notebooks	7
2.6 RMarkdown	8
2.7 Jupyter in Docker	8
2.8 Programmatische Orchestratie en Docker SDK's	9
2.9 Platform-onafhankelijke IPC: Sockets en Named Pipes	10
2.10 Besluit	10
3 Methodologie	12
3.1 Eerste fase: Probleemanalyse	12
3.2 Tweede fase: Requirement-analyse	12
3.3 Derde fase: Literatuurstudie en technologische verkenning	13
3.4 Vierde fase: Ontwerp en selectie van de oplossingsrichting	13
3.5 Vijfde fase: Proof-of-concept en evaluatie	13
4 Architectuur	14
4.1 Database laag	14
4.2 API laag	15
4.3 Jupyter laag	15
4.4 Besluit	16
5 Huidige staat van documentatie	17
5.1 Lessen uit het legacy project	17
5.2 Huidige aanpak in EMAN-web: technische README's	17
5.3 XML-commentaren en de limieten van DocFX	18
5.4 De "Single Source of Truth" ontbreekt	18

5.5	Besluit	18
6	Complexe onderdelen van EMAV	20
6.1	Domeincomplexiteit: De kloof tussen wetenschap en implementatie .	20
6.2	Hybride databeheer en persistentiestrategieën	21
6.3	De "Legacy Bridge" en technische schuld	21
6.4	Container-orchestratie en omgevingsbeheer	22
6.5	Repository Pattern	23
6.6	Command Query Responsibility Segregation (CQRS)	24
6.7	Mediator Pattern	24
6.8	Strategy Pattern	25
6.9	Factory Pattern	26
6.10	Facade Pattern.	26
6.11	Dependency Injection en Inversion of Control	26
6.12	Adapter Pattern	27
6.13	Decorator Pattern en Result Pattern	27
6.14	Clean Architecture	28
6.15	Besluit: Een synergetische oplossing voor gelaagde complexiteit . . .	28
7	Tekortschietsing van documentatie	30
7.1	Het probleem van "Documentation Drift"	30
7.2	Technische focus versus domeinfocus	31
7.3	Gebrek aan interactiviteit en executabiliteit	31
7.4	Besluit: De barrière van statische kennisoverdracht	32
8	Alternatieve vormen van documentatie	33
8.1	Het fundament: Literate Programming	33
8.2	Computationele notebook-platforms.	34
8.2.1	Jupyter-notebooks.	34
8.2.2	Quarto.	34
8.2.3	Org-mode & babel	35
8.3	De Techniek achter de Verweving: C# Interop in Notebooks.	35
8.4	Interactieve documentatie als leer- en validatieplatform	35
8.5	Besluit: Evaluatie van de alternatieven	36
9	Veilig experimenteren	37
9.1	Technische isolatie via Docker-sandboxing	37
9.2	Data-integriteit en lokale data-clonering	38
9.3	De weg van experiment naar productie: gecontroleerde promotie . . .	38
9.4	Besluit: Vangrails voor innovatie	38
10	Nauwere verweving tussen code & documentatie	40
10.1	Minder communicatie-overhead en ruis.	40
10.2	Documentatie als "Single Source of Truth"	40

10.3	Conclusie: een strategische transformatie.	41
11	Proof of Concept: hybride implementatiestrategie	42
11.1	Infrastructuur: docker-orchestratie	42
11.1.1	Geautomatiseerde DLL-collectie en versiebeheer	42
11.1.2	Programmatisch beheer via Docker.DotNet	43
11.1.3	Log-parsing en dynamische token-extractie.	43
11.2	Jupyter-container configuratie	43
11.2.1	De .NET 10.0 Runtime en Python-omgeving	44
11.2.2	Automatische Initialisatie via IPython	44
11.3	Strategie voor Granulaire Implementatie-Switches.	44
11.3.1	Het Switcheable-concept per Module	44
11.3.2	Realisatie via het Factory-patroon.	45
11.3.3	De Python-Interop in Actie	45
11.4	Veiligheid en Validatie	45
11.5	Resultaten en Beperkingen	46
11.6	Conclusie van de PoC	46
12	Conclusie	48
A	Onderzoeksvoorstel	50
A.1	Inleiding	50
A.2	Literatuurstudie	52
A.2.1	Literate programming	52
A.2.2	Org-mode Babel	52
A.2.3	Jupyter notebook	53
A.2.4	RMarkdown	53
A.2.5	Tussenbesluit	54
A.3	Methodologie	54
A.3.1	Eerste fase	54
A.3.2	Tweede fase	55
A.3.3	Derde fase	55
A.3.4	Vierde fase	55
A.3.5	Vijfde fase.	55
A.4	Verwacht resultaat, conclusie	56
A.4.1	Probleem- en domeinanalyse	56
A.4.2	Literatuurstudie & technologieverkenning	56
A.4.3	Proof-of-Concept ontwikkeling	56
A.4.4	Evaluatie en validatie	56
B	Codefragmenten	57
B.1	C# Implementatie details.	57
B.2	Container Configuratie	57

Bibliografie

63

Lijst van figuren

11.1 Architecturaal overzicht van de Proof of Concept met C#-orchestratie en hybride Python-integratie.	47
---	----

Lijst van tabellen

Lijst van codefragmenten

2.1	pythonnet officieel codefragment	5
2.2	C# code fragment	5
2.4	docker build image commando	9
2.3	Dockerfile voor Jupyter-notebook in docker	9
2.5	startup.py bij dockerfile	10
11.1	Factory voor het dynamisch wisselen tussen C# en Python implemen- taties.	45
11.2	C#-methode die via Python.NET een extern script aanroept.	46
11.3	Voorbeeld van een Python-script voor experimentele modelaanpas- singen.	46
B.1	pythonnet codefragment	58
B.2	Implementatie van de SetupDll-methode.	59
B.3	Aanmaken en opstarten van de Jupyter Docker-container.	60
B.4	Extractie van de Jupyter-token en poort uit logs.	61
B.5	Dockerfile voor de Jupyter-omgeving.	61
B.6	Startup-script voor IPython.	62

1

Inleiding

In onderzoeksprojecten worden berekeningen vaak eerst uitgewerkt door onderzoekers en pas daarna omgezet naar software. Dat is logisch: onderzoekers kennen het domein en de formules, terwijl ontwikkelaars instaan voor de technische uitwerking. Toch kan die verdeling ook voor problemen zorgen. Deze bachelorproef onderzoekt dat probleem binnen de context van ILVO en het EMAV-project. Bij ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) worden rekenmodellen gebruikt om onder andere milieu-impact en emissies binnen de landbouwsector te berekenen. Zulke modellen vertrekken vanuit wetenschappelijke kennis, formules en aannames die door onderzoekers worden opgesteld. Wanneer een model in een toepassing gebruikt moet worden, volstaat een Excel-bestand of los document meestal niet meer.

De berekeningen moeten dan worden omgezet naar software die betrouwbaar, onderhoudbaar en bruikbaar is voor eindgebruikers. In de onderzochte context gebeurt dat binnen een .NET-omgeving, met C# als belangrijkste programmeertaal.

De onderzoekers die met de modellen werken, zijn niet noodzakelijk vertrouwd met C#. Ze zijn vaak wel vertrouwd met de achterliggende formules, de parameters en de betekenis van de resultaten. Daardoor ontstaat een praktische vraag: hoe kan de software zo worden ingericht dat onderzoekers de berekeningen beter kunnen volgen en eventueel zelf kleine experimenten kunnen uitvoeren?

In de praktijk ontstaat er een afstand tussen de wetenschappelijke versie van het model en de technische implementatie ervan. Onderzoekers leveren formules, parameters of voorbeeldberekeningen aan. Ontwikkelaars vertalen die vervolgens naar C#-code, koppelen ze aan databanken en verwerken ze in de rest van de applicatie.

Dat werkt zolang het model stabiel blijft. Zodra een onderzoeker een formule wil aanpassen, een parameter wil testen of een alternatief scenario wil vergelijken, is

er opnieuw technische hulp nodig. De onderzoeker kan de berekening meestal niet rechtstreeks uitvoeren in de softwarecontext waarin het model echt gebruikt wordt. Omgekeerd is het voor ontwikkelaars niet altijd eenvoudig om de wetenschappelijke bedoeling achter elke berekeningsstap te blijven volgen.

De kern van het probleem is dat de berekeningslogica na verloop van tijd moeilijk zichtbaar wordt voor de onderzoeker. De code voert het model wel uit, maar de wetenschappelijke redenering zit verspreid over documentatie, broncode, overleg en voorbeeldbestanden. Daardoor kan de toepassing voor onderzoekers aanvoelen als een black box.

Een bijkomend probleem is dat documentatie snel achterloopt op de implementatie. Wanneer code wijzigt maar de uitleg niet mee aangepast wordt, ontstaat "documentation drift". Dat maakt het moeilijker om na te gaan of de software nog overeenkomt met het model zoals onderzoekers het bedoelen. Deze bachelorproef onderzoekt daarom hoe code, documentatie en experimenteeruimte dichter bij elkaar gebracht kunnen worden.

Deze bachelorproef probeert niet het volledige EMAN-project te herwerken. Ook het volledige emissiemodel zelf wordt niet opnieuw ontworpen. De scope ligt bij een proof-of-concept waarin wordt nagegaan hoe C#-logica beschikbaar gemaakt kan worden in een Python-omgeving en hoe een alternatieve implementatie getest kan worden zonder de hoofdapplicatie rechtstreeks aan te passen.

De nadruk ligt dus op de technische haalbaarheid en op de bruikbaarheid van de werkwijze voor onderzoekers. Beveiliging, performantie en integratie in een productieomgeving worden besproken waar ze nodig zijn om de PoC (proof-of-concept) te beoordelen, maar ze worden niet volledig uitgewerkt.

Vanuit deze problematiek is de volgende centrale onderzoeksvraag geformuleerd: *Hoe kunnen we de kloof tussen IT-logica en wetenschappelijke analyse verkleinen, zodat onderzoekers zonder diepgaande programmeerkennis inzicht krijgen in hun modellen en er zelf mee kunnen experimenteren?*

Om deze hoofdvraag te beantwoorden, worden de volgende deelvragen onderzocht:

1. Welke onderdelen van de bestaande code en architectuur maken het moeilijk voor onderzoekers om de berekeningen te volgen?
2. Waarom volstaat de huidige documentatie niet om het model en de implementatie samen duidelijk te maken?
3. Welke vormen van interactieve documentatie of Jupyter-notebooks zijn geschikt om berekeningen begrijpelijker te maken?
4. Hoe kan een onderzoeker veilig experimenteren met een alternatieve implementatie zonder de stabiliteit van de applicatie te schaden?

5. Hoe kan een proof-of-concept aantonen dat C#-logica en Python-code samen gebruikt kunnen worden binnen deze context?

Het doel van dit onderzoek is om een haalbare werkwijze te verkennen waarmee onderzoekers meer inzicht krijgen in de berekeningen, zonder dat de bestaande software volledig herbouwd moet worden. Daarvoor wordt een proof-of-concept uitgewerkt. De focus ligt niet alleen op documentatie, maar ook op de vraag hoe onderzoekers op een gecontroleerde manier met de berekeningen kunnen experimenteren.

De belangrijkste doelstellingen zijn:

- Architecturale scheiding: nagaan hoe domeinlogica binnen een .NET-project duidelijker gescheiden kan worden van infrastructuur zoals databanken en API's.
- Hybride integratie: onderzoeken hoe C#-modules via pythonnet beschikbaar gemaakt kunnen worden in een Python-omgeving.
- Experimenteerruimte: bekijken hoe een onderzoeker een alternatieve implementatie kan testen zonder de officiële toepassing aan te passen.
- Evaluatie: beoordelen in welke mate de PoC helpt om berekeningen begrijpelijker en beter testbaar te maken.

De rest van deze bachelorproef volgt de deelvragen. Eerst wordt in de stand van zaken bekeken welke bestaande technieken en werkwijzen relevant zijn, zoals Python.NET, computationele notebooks, literate programming en containerisatie. Daarna licht de methodologie toe hoe het probleem en de mogelijke oplossing onderzocht werden.

Vervolgens wordt de architectuur van het proof-of-concept besproken, met aandacht voor de databank, API en notebooks. Daarna volgen hoofdstukken over de huidige documentatie, de complexiteit van het project, veilig experimenteren en de verweving tussen code en documentatie. Ten slotte wordt de proof-of-concept besproken en wordt in de conclusie teruggekoppeld naar de onderzoeksvraag en deelvragen.

2

Stand van zaken

In dit hoofdstuk worden de technologische bouwstenen onderzocht die de basis vormen voor de voorgestelde hybride strategie. Dit begint bij een analyse van bridge-technologieën voor taal-interoperabiliteit, zoals Python.NET en IronPython, om de interactie tussen C# en Python mogelijk te maken. Vervolgens wordt het concept van *Literate Programming* uitgediept, waarbij de evolutie van Knuths oorspronkelijke principes naar moderne computationele notebooks (zoals Jupyter en RMarkdown) centraal staat. Ten slotte worden de infrastructurele aspecten en design patterns besproken die nodig zijn om deze technologieën veilig en schaalbaar te orkestreren. Deze theoretische en technische verkenning vormt het noodzakelijke fundament voor de architecturale uitwerking die in het volgende hoofdstuk aan bod komt.

2.1. Pythonnet

Naast de bedoeling om de code te documenteren, is het ook de bedoeling om de codebase die in C# is geschreven te laten interageren met python. Op basis van gesprekken met medewerkers kennen de onderzoekers geen C# maar wel python. Omdat zij groten deels beslissen wat de berekeningen zijn, kunnen ze dat wel in python zetten. Pythonnet zorgt voor een integratie van Python Software Foundation (2026) engine en de .NET runtime. Het compileert geen python naar IL (Intermediate Language), een assembler achtige taal die wordt gebruikt als een brug tussen high-end C# code en low-end machine code. Het kan dan CLR (Common Language Runtime) services aanspreken, bestaande python code gebruiken en C-API extensies gebruiken met de normale uitvoer snelheden van python. Het zal automatisch op windows het .NET Framework kiezen en op Linux en MacOS zal het Mono Project (2026) gebruiken.

De CLR (Microsoft, 2026) is een runtime om .NET code uit te voeren. De talen die

```
1 import clr
2 from System import String
3 from System.Collections import *
```

Codefragment 2.1: Codefragment van de website

```
1 using Python.Runtime;
2
3 var filePath = ""; // pad naar python bestand
4 PythonEngine.Initialize();
5 using (Py.GIL())
6 {
7     sys.path.insert(0, folder);
8     string moduleName = Path.GetFileNameWithoutExtension(filePath);
9     try
10     {
11         dynamic module = Py.Import(moduleName);
12     }
13 }
```

Codefragment 2.2: C# code die een python file inleest en beschikbaar maakt voor uitvoer

op deze runtime kunnen worden uitgevoerd zijn C#, VB.NET en F#. Andere talen kunnen ook worden uitgevoerd via de .NET runtime, maar worden niet direct ondersteund door Microsoft (2026). De compiler vormt de code om naar IL, erachter voert de CLR een JIT-compilatie (Just In Time compilatie) uit om de code naar machine code te vertalen.

Een runtime is de softwarelaag die een programma uitvoert en tijdens die uitvoering diensten aanbiedt zoals geheugenbeheer, typecontrole, exception handling en het laden van code. In het geval van Microsoft .NET is de Common Language Runtime (CLR) de runtime die beheerde code omzet naar machinecode en die uitvoering coördineert. (Microsoft, 2026)

Via pythonnet (contributors of the Python.NET project, 2026) kan python dll's lezen, importeren en gebruiken, zoals in het codefragment B.1 dat een Windows Forms Applicatie maakt vanuit python.

Het toffe van pythonnet is dat men naast C# python kan aanroepen, het ook omgekeerd kan.

2.2. IronPython

IronPython (.NET foundation, 2026) is een open source implementatie van de Python programmeer taal die is geïntegreerd met .NET. IronPython kan zoals pythonnet .NET dll's aanspreken via de CLR en kan andere python code ook gebruiken. In tegenstelling tot pythonnet kan IronPython niet aangeroepen worden vanuit .NET gebaseerde talen.

Een groot nadeel van IronPython is dat het niet de nieuwste versie van python gebruikt, waarin dat pythonnet werkt tot en met versie 3.13, heeft IronPython support voor versie 2.7 en 3.4. (.NET foundation, 2026)

2.3. Literate programming

Literate programming (Knuth, g.d.) werd geïntroduceerd door Knuth (1992) als een methode waarbij code en documentatie 1 geïntegreerd geheel vormen, eerder dan gescheiden artefacten. De bedoeling is dat er 1 bestand is met de code en de documentatie errond. De tools halen de code eruit en creëren een uitvoerbaar bestand en een ander bestand met de documentatie, dat kan een html web pagina zijn, of een $\text{\LaTeX}$ bestand / pdf. Het kan ook een Markdown bestand zijn.

In plaats van 'code with comments' vertrekt literate programming vanuit het idee dat een programma in de eerste plaats een verhaal is dat aan een menselijke lezer moet worden uitgelegd, waarbij uitvoerbare code een secundaire afgeleide vorm is. (Knuth, 1992) Sindsdien zijn er talrijke systemen ontstaan die Knuths principes toepassen in een moderne ontwikkelingsomgeving.

Hoewel Knuths oorspronkelijke WEB en CWEB nog weinig gebruikt worden in de hedendaagse softwarepraktijk, leven zijn ideeën voort in computationele notebooks, literate programminglibraries en research pipelines. Tools zoals Jupyter Notebooks, Org-mode Babel en Quarto implementeren een moderne interpretatie van Knuths 'weave' en 'tangle'-proces, waarin broncode zowel als uitvoerbaar script en als leesbare documentatie dient.

De execution haalt de source code eruit en zorgt voor een uitvoerbaar bestand. De Weave operatie zorgt er voor dat de documentatie uit het source bestand wordt gehaald en die in het gewenste formaat zet. De Tangle operatie verspreidt de source code in de respectievelijke bestanden zelf, niet direct uitvoerbaar. (Knuth, 1992)

2.4. Org-mode Babel

Org mode is een 'major mode' voor GNU Emacs. Org mode kan voor heel veel gebruikt worden: van notities nemen tot todo-lists, computational notebooks en literate programming. (Dominik & Schulte, 2025)

In Org Mode met Babel zijn er 'src-blocks':

1 * Src-blocks in org-mode

```
2  #+BEGIN_SRC python :results verbatim
3      nummer: int = 10
4      return nummer
5  #+END_SRC
6
7  #+RESULTS:
8  : 10
```

Hierin kan code worden uitgevoerd terwijl er tekst rond staat. Dit is het idee van literate programming. De meeste 'cellen' in org-mode hebben hun eigen omgeving, maar er bestaan manieren om dit te vermijden. Babel verbetert de src-blocks door een paar zaken te voorzien:

- interactieve en programma executie van code blokken
- code blokken die functies met parameters hebben, kunnen andere code blokken accepteren en oproepen, en
- bestanden exporteren voor literate programming

2.5. Jupyter-notebooks

Jupyter-notebooks zijn gemaakt voor python. Deze notebooks worden door vele mensen gebruikt en in verschillende omgevingen. Dit kan gaan van een student die python leert en zijn notities er bij wil schrijven tot een volledige data analyse door professionele onderzoekers. Dit is een standaard voor computationeel onderzoek en datascience (Project Jupyter, [2025](#)). Het werkt met verschillende 'cellen': de markdown cellen en de python cellen. De python cellen verbinden met een virtuele omgeving en voeren hierop telkens hun code uit. Aangezien ze werken op dezelfde omgeving zijn de cellen verbonden aan elkaar. Het grootste verschil met org-mode is dat een .org bestand een plain tekst bestand is. Het is mogelijk om dit bestand te bekijken in eender welke editor. Jupyter-notebooks niet. Je hebt een editor nodig die Jupyter-notebooks ondersteunt. Het bekendste is VsCode van Microsoft. Er bestaat ook de officiële command en app van jupyter zelf die een server opstart en dan in de browser beschikbaar is.

Maar Jupyter-notebooks hebben zeker hun minpunten. Rule e.a. ([2018](#)) tonen aan dat Jupyter-notebooks niet-lineaire uitvoer geschiedenis hebben, wat reproduceerbaarheid ernstig bemoeilijkt. Ook omdat de cellen in een willekeurige volgorde kunnen worden uitgevoerd, moet er opgelet worden met de run volgorde. De notebooks hebben ook een onzichtbare global state. Het zijn json bestanden, dus versie controle beheer kan hier ook moeilijk mee doen. Aangezien alles ook in 1 bestand zit, kan het snel het groot bestand zijn.

2.6. RMarkdown

RMarkdown (RStudioPBC, 2020) is ook een computational notebook zoals Jupyter Notebooks, maar het ondersteunt meerdere talen. Het volgt een structuur die sterk aansluit bij de principes van literate programming. Een .Rmd-bestand wordt verwerkt door knitr (Xie e.a., 2025), een transparante engine voor het genereren van dynamische rapporten in R, ontwikkeld binnen de softwareomgeving voor statistische computing R (The R Foundation, g.d.).

Knitr voert alle code blokken uit en maakt een nieuw Markdown bestand aan met alle code en hun uitkomsten. Het gemaakte Markdown bestand wordt dan geëvalueerd door pandoc (MacFarlane, 2025), die dan het gewenste product maakt. Vb:

```
1 ---
2 title: "viridis demo"
3 output: html_document
4 ---
5
6 ```{r include=FALSE}
7 library(viridis)
8 ```
9 The code below ...
10
11 ## Colors
12 ```{r}
13 image(volcano, col = viridis(200))
14 ```
```

2.7. Jupyter in Docker

In een Jupyter-notebook kan men code uitvoeren die direct op de server kan worden uitgevoerd, dit is perfect voor een hacker om de volledige infrastructuur neer te halen via een RCE (Remote Code Execution). Hierbij kan een hacker, eens toegang tot de Jupyter-notebook, gevaarlijke code of commando's uitvoeren om admin access te krijgen via python. Eens dat gebeurt kan de hacker de server besturen en zitten ze in het intern netwerk. Als dit in een bedrijf gebruikt wordt op een server moet er dus een zware beveiliging op zitten. Een goede start is isolatie.

Aangezien er dll's nodig gaan zijn en mogelijks C# code gaat moeten uitvoeren, wordt er gestart in de DockerFile vanuit een Microsoft runtime image.

```
1 docker build -t jupyter-image .
```

Codefragment 2.4: docker build image commando van de dockerfile in de huidige directory

```
1 FROM mcr.microsoft.com/dotnet/runtime:8.0
2 RUN apt-get update && apt-get install -y python3 python3-pip
3 RUN pip3 install jupyterlab \
4         pythonnet \
5         ipykernel \
6         --break-system-packages
7 WORKDIR /app
8 COPY dlls /app/dlls
9 COPY notebooks /app/notebooks
10 COPY startup.py /root/.ipython/profile_default/ipython_config.py
11 EXPOSE 8888
12 CMD ["jupyter", "lab", "--ip=0.0.0.0", "--allow-root",
      "--no-browser", "--notebook-dir=/app/notebooks"]
```

Codefragment 2.3: Dockerfile voor Jupyter-notebook in docker

Deze dockerfile downloadt de nodige packages, kopieert de dlls in de dlls folder en de notebooks uit de notebook folder en start de Jupyter-notebook server.

De dockerfile kan gebuild worden door dit commando: Hierdoor is deze image beschikbaar op de server en kan die gevonden worden door de C# code.

De startup.py is een python bestand dat bij het maken van de docker image code uitvoert tijdens dat de kernel start. (IPython Development Team, [g.d.](#)) Hierdoor kunnen er dll's geladen worden voordat de gebruiker toegang heeft en veel boilerplate mag weglaten.

2.8. Programmatische Orchestratie en Docker SDK's

Voor het beheren van containers in een hybride omgeving is er de keuze tussen het aanroepen van de Docker Command-Line Interface (CLI) via shell-scripts of het gebruik van een native Software Development Kit (SDK) zoals `Docker.DotNet`.

Hoewel de CLI eenvoudiger is voor ad-hoc taken, biedt een SDK aanzienlijke voordelen op het vlak van robuustheid en beveiliging (Richardson, 2018). Programmatische aansturing via een SDK voorkomt "shell injection" kwetsbaarheden, aangezien parameters als getypeerde objecten worden doorgegeven in plaats van als tekstuele commando's. Bovendien laat een SDK toe om directe feedback van de Docker-daemon te verwerken via rijke exceptie-objecten, wat essentieel is voor foutafhandeling in een multi-user omgeving zoals die van ILVO.

```
1 c = get_config()
2 c.InteractiveShellApp.exec_lines = [
3     'import sys',
4     'import pythonnet',
5     'pythonnet.load("coreclr")',
6     'import clr',
7     'sys.path.append("/app/dlls")',
8     'clr.AddReference("<dll naam zonder '.dll'>")',
9     'from <imported namespace> import *'
10 ]
```

Codefragment 2.5: startup.py dat wordt geladen tijdens de kernel opstart van de Jupyter-notebook kernel

2.9. Platform-onafhankelijke IPC: Sockets en Named Pipes

De communicatie tussen de orchestrerende applicatie en de Docker-daemon vindt lokaal plaats via Inter-Process Communication (IPC). De gebruikte techniek is afhankelijk van het onderliggende besturingssysteem (Docker Inc., [2026b](#)).

Op Linux-gebaseerde systemen zijn Unix Sockets de standaard. Deze bieden een hoogperformant communicatiekanaal met minimale overhead, omdat zij de volledige netwerkstack omzeilen. Op Windows-omgevingen wordt echter gebruikgemaakt van Named Pipes (`\\./pipe/docker_engine`). Hoewel de technologische implementatie verschilt, abstraheren moderne SDK's deze complexiteit weg, waardoor dezelfde C#-codebase zowel op een Windows-ontwikkelmachine als op een Linux-productieserver kan opereren.

2.10. Besluit

Uit de verkenning van de huidige stand van zaken blijkt dat de technologische fundamenteën voor een nauwere verweving tussen IT-implementatie en wetenschappelijk onderzoek ruimschoots aanwezig zijn. De combinatie van krachtige bridge-technologieën zoals Python.NET en de volwassenheid van computationele Jupyter-notebooks biedt een realistisch pad om de gecompileerde .NET-wereld toegankelijk te maken voor onderzoekers zonder de robuustheid van de backend te ondermijnen.

Daarbij is vastgesteld dat de inzet van containerisatie via Docker niet louter een infrastructurele keuze is, maar een essentiële voorwaarde om de risico's van Remote Code Execution te beheersen en een consistente, reproduceerbare analyse-omgeving te garanderen. De besproken design patterns (zoals Factory en Proxy) vormen hierbij de architecturale lijm die ervoor zorgt dat de integratie tussen beide

talen gestructureerd en onderhoudbaar verloopt.

De technologische bouwstenen die in dit hoofdstuk zijn geanalyseerd, worden in het volgende hoofdstuk samengebracht. Daar zal worden toegelicht hoe deze componenten zijn georkestreerd in een specifieke architectuur die de basis vormt voor de Proof of Concept binnen het EMAN-project.

3

Methodologie

Binnen een concrete casus bij het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek (ILVO) wordt een toegepast, onderwerpgericht onderzoeksofzet gevolgd. De focus ligt op het analyseren van de huidige samenwerking tussen onderzoekers en softwareontwikkelaars rond een .NET-gebaseerd rekenmodel, en op het uitwerken van een haalbare oplossingsrichting die deze samenwerking ondersteunt. Het onderzoekstraject is opgedeeld in vijf fasen, waarbij de feitelijke uitvoering organisch evolueerde op basis van voortschrijdend inzicht.

3.1. Eerste fase: Probleemanalyse

In de eerste fase werd het probleemdomein in kaart gebracht door middel van een verkennend gesprek met de copromotor en een grondige analyse van de bestaande documentatie. Aanvankelijk werd de problematiek gedefinieerd als een puur documentatieprobleem, maar gaandeweg werd duidelijk dat de kern van het probleem lag in de afstand tussen de wetenschappelijke logica en de technische implementatie in C#. Door concrete use-cases van de EMAN-applicatie te bestuderen, werd vastgesteld dat onderzoekers beperkt worden door de black-boxaard van de software.

3.2. Tweede fase: Requirement-analyse

De tweede fase richtte zich op het identificeren van de noden van zowel onderzoekers als ontwikkelaars. Voor dit onderzoekstraject is ervoor gekozen om de vereisten primair af te leiden uit de expertise van de copromotor, de analyse van eerdere samenwerkingsmomenten en de functionele behoeften van het EMAN-project. Deze aanpak bood een gerichte basis om de technische requirements te definiëren, waarbij de focus lag op transparantie, autonome experimenteerruimte en het voorkomen van documentation drift, met behoud van de stabiliteit van de

productieomgeving.

3.3. Derde fase: Literatuurstudie en technologische verkenning

In de derde fase werd gezocht naar technologische oplossingen die de kloof tussen code en documentatie kunnen overbruggen. Er werd een samenvatting opgesteld van de "Stand van zaken", waarbij concepten zoals Literate Programming en interactieve notebooks (zoals Jupyter en Quarto) werden onderzocht. Deze fase was cruciaal om te bepalen welke technologieën compatibel zijn met de bestaande .NET-architectuur van ILVO en hoe deze ingezet kunnen worden om berekeningen inzichtelijk te maken.

3.4. Vierde fase: Ontwerp en selectie van de oplossingsrichting

Op basis van de technologische verkenning en een diepgaande analyse van de EMAV-broncode, werd in de vierde fase een specifieke oplossingsrichting gekozen. Er werd besloten om Python te integreren binnen de .NET-context. Het ontwerp voorzag in een 'switch'-mechanisme waarmee binnen de applicatie gewisseld kan worden tussen de officiële C#-implementatie en een experimentele Python-versie. Deze hybride aanpak werd geselecteerd omdat het onderzoekers de vertrouwde Python-omgeving biedt zonder de robuustheid van de C#-rekenmodules te ondermijnen.

3.5. Vijfde fase: Proof-of-concept en evaluatie

In de laatste fase werd de voorgestelde oplossing gerealiseerd in een proof-of-concept. Dit prototype diende als validatie voor de technische haalbaarheid van de Python-integratie en de inzet van notebooks als interface. De evaluatie vond plaats door de deelvragen van de bachelorproef stapsgewijs te beantwoorden op basis van de bevindingen uit de PoC. Hierbij werd getoetst in welke mate de voorgestelde werkwijze effectief bijdraagt aan de autonomie van de onderzoeker en de traceerbaarheid van de berekeningen, wat de basis vormde voor de uiteindelijke conclusies.

4

Architectuur

Om de voorgestelde oplossing voor de kloof tussen IT en onderzoek te valideren, is een specifieke architectuur ontworpen voor de Proof-of-Concept (PoC). In dit hoofdstuk wordt de technische opbouw van dit ecosysteem toegelicht, waarbij de focus ligt op het creëren van een veilige en flexibele omgeving waarin .NET-logica ontsloten wordt voor wetenschappelijke analyse. Deze PoC-architectuur dient als blauwdruk voor een werkwijze waarbij robuuste software-engineering hand in hand gaat met interactieve experimenteerruimte.

De architectuur is opgedeeld in drie complementaire lagen die elk een specifieke rol spelen in het ondersteunen van de onderzoeker. Ten eerste is er de database-laag, die via gescheiden databronnen een veilige experimenteerruimte biedt zonder de operationele systemen te belasten. Ten tweede fungeert de API-laag als de centrale gateway voor businesslogica en data-uitwisseling. Ten slotte vormt de Jupyter-laag het hart van de interactieve documentatie, waar onderzoekers via Python rechtstreeks kunnen interageren met de onderliggende .NET-modules.

In de volgende secties worden deze lagen technisch uitgediept, waarbij de nadruk ligt op de wijze waarop isolatie, veiligheid en interoperabiliteit binnen de PoC zijn gerealiseerd.

4.1. Database laag

De databases bestaan uit een continu geüpdatete kopie van de productieomgeving, zodat onderzoekers steeds over actuele data beschikken zonder de operationele systemen te beïnvloeden.

De database-instanties zijn uitsluitend bedoeld voor het uitlezen van data; wijzigingen aan de data zijn niet toegestaan. Om de veiligheid en integriteit te waarborgen is er geen directe connectie met de productie-databases. In plaats daarvan wordt gewerkt met gescheiden databronnen binnen een gecontroleerde omgeving.

Voor toegang tot de data wordt gebruikgemaakt van een gebruiker met uitsluitend read-only rechten. Deze gebruiker heeft enkel SELECT-permissies op vooraf gedefinieerde views of schema's en beschikt niet over write- of DDL-rechten. Daarnaast worden extra restricties toegepast, zoals statement timeouts en limieten op het aantal rijen per query, om misbruik en overbelasting te voorkomen.

Indien de belasting van de productieomgeving gevoelig is, kan er gebruik worden gemaakt van een read-replica. Deze replica dient als databron voor bijvoorbeeld Jupyter-notebook-omgevingen, zodat analytische workloads geen impact hebben op de performantie van de operationele systemen.

Verder kunnen er specifieke views of API-klare datasets worden voorzien, zodat gebruikers nooit rechtstreeks met onderliggende tabellen werken. Dit zorgt voor extra abstractie, verhoogde veiligheid en een stabielere datastructuur voor analyse.

4.2. API laag

De communicatie tussen de verschillende componenten verloopt via een robuuste API-laag. Deze laag bevindt zich in het afgesloten Docker-netwerk, zodat interne services zoals Jupyter-containers (zie Hoofdstuk 11) veilig kunnen communiceren zonder blootstelling aan het publieke internet.

De API fungeert als de gateway voor alle data-operaties en berekeningsverzoeken. Door gebruik te maken van gestandaardiseerde REST-principes en JSON als uitwisselingsformaat, wordt een hoge mate van interoperabiliteit gegarandeerd tussen de .NET-backend en de diverse cliënt-omgevingen.

4.3. Jupyter laag

Voor data-analyse en berekeningen wordt gebruikgemaakt van Jupyter Notebook-omgevingen die draaien binnen Docker-containers. Deze containers worden opgebouwd via een custom Docker image waarin zowel Jupyterlab als pythonnet geïnstalleerd zijn.

Binnen deze omgeving worden vooraf gecompileerde .NET DLL's toegevoegd, die gebruikt kunnen worden om complexe berekeningen of domeinspecifieke logica uit te voeren. Via configuratie in sitecustomize.py worden deze DLL's automatisch geladen bij het opstarten van de Python runtime, zodat ze onmiddellijk beschikbaar zijn binnen notebooks.

De containers draaien onder een niet-root gebruiker om de veiligheid te verhogen en het risico op privilege-escalatie te beperken.

Om misbruik en overbelasting te voorkomen, worden resourcebeperkingen ingesteld op de containers, zoals limieten op geheugen, CPU-gebruik en het aantal processen. Daarnaast wordt het bestandssysteem zoveel mogelijk read-only gehouden en worden extra beveiligingsmaatregelen toegepast, zoals het verwijderen van Linux capabilities (cap-drop) en het inschakelen van no-new-privileges.

Ook op netwerkniveau worden restricties toegepast: de Jupyter containers kunnen enkel communiceren met de API-laag (de Razor server) binnen het Docker-netwerk. Toegang tot het internet of andere containers wordt geblokkeerd, zodat de omgeving strikt gecontroleerd blijft.

Door gebruik te maken van .NET DLL's binnen Python-notebooks wordt hergebruik van bestaande businesslogica mogelijk, terwijl onderzoekers toch flexibel kunnen werken in een Python-gebaseerde analyseomgeving.

Ten slotte wordt de volledige setup lokaal getest om te verifiëren dat notebooks correct de DLL's kunnen laden via pythonnet, en dat API-requests (GET) succesvol uitgevoerd kunnen worden.

4.4. Besluit

De gelaagde architectuur van de Proof-of-Concept biedt een gestructureerd antwoord op de uitdaging om complexe .NET-software toegankelijk te maken voor wetenschappelijk onderzoek. Door een strikte scheiding aan te brengen tussen de data-opslag, de logica-ontsluiting via de API en de interactieve analyse-omgeving in de Jupyter-laag, wordt een ecosysteem gecreëerd dat zowel veilig als functioneel is.

Deze opzet garandeert dat de integriteit van de centrale EMAN-applicatie gewaarborgd blijft, terwijl onderzoekers de nodige autonomie krijgen om hypothesen te toetsen en modellen te valideren. De inzet van containerisatie en gecontroleerde data-cloning vormt hierbij de noodzakelijke beveiligingsschil. Hiermee is het architecturale fundament gelegd voor de feitelijke realisatie van de Proof-of-Concept, die in het volgende hoofdstuk in detail wordt beschreven.

5

Huidige staat van documentatie

Alvorens dieper in te gaan op de tekortkomingen van de huidige aanpak, is het noodzakelijk om de huidige staat van documentatie binnen het EMAV-web project te schetsen en deze te contrasteren met de situatie in de originele Windows Forms applicatie. Momenteel bevindt het project zich in een overgangsfase, waarbij de lessen uit het verleden de koers voor de toekomst bepalen.

5.1. Lessen uit het legacy project

In de oorspronkelijke EMAV-applicatie was de documentatie gefragmenteerd. Voor de onderzoekers bestond de documentatie voornamelijk uit lijvige Word-documenten en PDF-handleidingen die losstonden van de codebasis. Voor de programmeurs waren er pogingen ondernomen om complexe business-flows (zoals de kunstmest-flow) te documenteren in afzonderlijke Markdown-bestanden binnen de docs-map van de repository.

Deze aanpak leed echter aan een chronisch gebrek aan onderhoudbaarheid. In de praktijk fungeerden deze Markdown-bestanden als een soort "schaduw-code": ze bevatten grote blokken gekopieerde C#-logica met minimale tekstuele uitleg. Dit zorgde voor een enorme documentation drift: zodra een formule in de rekenkern werd aangepast, werd de corresponderende uitleg in de documentatiemap vergeten. Het resultaat was een systeem waarbij de documentatie eerder voor verwarring zorgde dan voor verduidelijking, wat de afhankelijkheid van de oorspronkelijke ontwikkelaars alleen maar vergrootte.

5.2. Huidige aanpak in EMAV-web: technische README's

In de nieuwe EMAV-web applicatie is er een bewuste verschuiving merkbaar naar Documentation as Code. De primaire vorm van documentatie voor de ontwikkelaars bestaat momenteel uit Markdown-bestanden, zoals de centrale README.md

in de root van de repository. Deze bestanden zijn veel directer gekoppeld aan het ontwikkelproces: ze bevatten TODO NEXT-lijsten, instructies voor database-migraties en architecturale diagrammen in Mermaid-formaat.

Hoewel dit een aanzienlijke verbetering is voor de IT-onboarding, blijft de kloof met de onderzoekers van ILVO. De informatie in deze bestanden is nagenoeg uitsluitend technisch van aard. Voor een onderzoeker die wil begrijpen hoe een emissiefactor tot stand komt, biedt een README met instructies over de `WebDbContext` migraties geen enkel inzicht. De documentatie is opgesteld voor en door ontwikkelaars, waardoor de wetenschappelijke essentie van het project nog steeds onderbelicht blijft.

5.3. XML-commentaren en de limieten van DocFX

Binnen de nieuwe C#-codebase wordt consequent gebruikgemaakt van `/// <summary>` XML-commentaren. Er zijn concrete plannen om DocFX te gebruiken om deze commentaren automatisch te publiceren als een statische website. Hoewel dit de technische transparantie verhoogt, lost het de fundamentele vraag van de onderzoeker niet op. XML-commentaren zijn gebonden aan de structuur van de klassen en methoden. Ze zijn uitstekend voor het beschrijven van input-parameters, maar ongeschikt voor het weergeven van de complexe, domeinoverschrijdende wetenschappelijke redeneringen die EMAV uniek maken.

5.4. De "Single Source of Truth" ontbreekt

De huidige realiteit is dat de eigenlijke levende documentatie van de wetenschappelijke modellen zich nog steeds buiten de broncode bevindt, in Excel-sheets en verslagen van commissies. Er is geen enkel mechanisme dat garandeert dat de formule in de Excel-sheet van de onderzoeker exact dezelfde is als de implementatie in `ILVO.EMAV.Core`. Deze versnippering is het kernprobleem dat in dit onderzoek wordt geadresseerd. De noodzaak voor een vorm van documentatie die zowel executabel is voor de machine als leesbaar voor de wetenschapper — zoals bij literate programming — komt direct voort uit deze historische en actuele tekortkomingen.

5.5. Besluit

De analyse van de huidige staat van documentatie leert dat er een fundamenteel onderscheid bestaat tussen documentatie voor softwareontwikkeling en documentatie voor wetenschappelijk onderzoek. Hoewel de overstap naar *Documentation as Code* en het gebruik van technische README's de efficiëntie voor IT-ontwikkelaars heeft verhoogd, blijven de wetenschappelijke modellen voor onderzoekers verborgen achter een technische abstractielaag.

De belangrijkste les uit deze inventarisatie is dat statische documentatievormen, ongeacht hun formaat, gedoemd zijn om te divergeren van de feitelijke implemen-

tatie (documentation drift). Zodra de wetenschappelijke logica en de technische code gescheiden werelden blijven, ontstaat een afhankelijkheid die innovatie en autonomie belemmert. Dit inzicht vormt de directe aanleiding om verder te kijken dan statische teksten en te zoeken naar een oplossing waarbij de documentatie de broncode niet slechts beschrijft, maar deze ook daadwerkelijk uitvoert. Dit brengt ons bij de complexiteit van het huidige project en de noodzaak voor een meer geïntegreerde aanpak, zoals besproken in de volgende hoofdstukken.

6

Complexe onderdelen van EMAV

Hoewel de overstap naar een modern webframework (.NET Razor Pages) veel van de technologische beperkingen van het oude Windows Forms-project wegneemt, blijft EMAV-web een applicatie met een aanzienlijke cognitieve last voor ontwikkelaars. In dit hoofdstuk wordt de gelaagde complexiteit van het project geanalyseerd, variërend van de wetenschappelijke domeinlogica tot de technische infrastructuur.

Achtereenvolgens wordt ingegaan op de kloof tussen wetenschappelijke modellen en software-implementatie, de uitdagingen van hybride databasebeheer en de problematiek van de “Legacy Bridge” bij het hergebruiken van oude rekenmodules. Tevens worden de infrastructurele implicaties van container-orchestratie besproken. Deze inventarisatie van uitdagingen vormt het noodzakelijke kader om te begrijpen waarom de architectuur van EMAV-web steunt op een uitgebreid frame van software-ontwerppatronen (design patterns) en de principes van Clean Architecture, die in de tweede helft van dit hoofdstuk in detail worden uitgediept.

6.1. Domeincomplexiteit: De kloof tussen wetenschap en implementatie

Het meest prominente obstakel bij het begrijpen en beheren van de codebase is niet zozeer dat de programmeurs de logica niet zouden vatten, zij zijn immers verantwoordelijk voor de vertaling van de wiskundige modellen naar werkende software, maar eerder de enorme drempel die de C#-implementatie opwerpt voor de onderzoekers zelf. In tegenstelling tot klassieke administratieve applicaties vormt de kern van EMAV een web van onderling afhankelijke wetenschappelijke formules die gebaseerd zijn op jarenlang landbouwkundig onderzoek.

Hoewel de code voor een ontwikkelaar technisch helder en logisch gestructureerd kan zijn, fungeert ze voor een onderzoeker vaak als een black box. De wetenschap-

pers, die de eigenlijke domeinexperts zijn, herkennen hun eigen wiskundige modellen vaak niet onmiddellijk terug in de abstracties van een objectgeoriënteerde taal. Dit zorgt voor een fundamenteel probleem in de levenscyclus van het project: wanneer een onderzoeker een formule wil finetunen of een nieuwe parameter wil testen, is hij volledig afhankelijk van de IT-afdeling. Het maken van zelfs de kleinste aanpassingen vereist een cyclus van overleg, implementatie en validatie door de programmeur, wat experimentatie en snelle iteratie ernstig belemmert. De complexiteit zit dus niet in de onbegrijpelijkheid van de code voor IT, maar in het gebrek aan autonomie voor de wetenschapper binnen een puur technische omgeving.

6.2. Hybride databeheer en persistentiestrategieën

Een ander complex onderdeel is de manier waarop EMAV omgaat met data. Het systeem maakt gebruik van een hybride architectuur waarbij verschillende database-technologieën naast elkaar worden gebruikt. Metadata over gebruikers, projecten en overzichten worden opgeslagen in een centrale SQL Server-database via Entity Framework Core. De ruwe inputgegevens voor berekeningen bevinden zich echter vaak in SQLite-bestanden die per project of per onderzoeksgroep worden beheerd.

Deze scheiding is noodzakelijk om onderzoekers de nodige flexibiliteit te bieden om met eigen datasets te werken, maar het introduceert aanzienlijke complexiteit in de datatoegangslaag. Ontwikkelaars moeten navigeren tussen verschillende database providers, rekening houden met inconsistente schema's en zorgen voor data-integriteit over de grenzen van de verschillende databases heen. Het beheer van deze persistentiestromen vereist een diepgaand inzicht in zowel de infrastructuurlaag als de domeinregels.

6.3. De "Legacy Bridge" en technische schuld

Hoewel EMAV-web als een nieuw project is gestart, is het onvermijdelijk dat er logica uit de oude applicatie is overgenomen. Legacy-systemen bevatten vaak bedrijfskritische kennis die niet zomaar mag verdwijnen; volgens Comella-Dorda e.a. (2000) zijn zulke systemen vaak tegelijk te belangrijk om weg te gooien en te kwetsbaar om zonder meer te wijzigen. Het hergebruiken van complexe, gevalideerde rekenmodules uit de Windows Forms-tijd is daarom essentieel om de wetenschappelijke consistentie te garanderen. Dit creëert echter een Legacy Bridge: een zone in de code waar moderne C#-conventies en verouderde programmeerpatronen met elkaar in conflict komen.

Het adapteren van deze oude logica naar een moderne, asynchrone Clean Architecture vereist veel plumbing code. Deze aanpak sluit aan bij oplopende modernisering: Seacord e.a. (2001) beschrijven dat modernisering vaak betekent dat legacy-componenten en vernieuwde componenten tijdelijk naast elkaar

moeten functioneren, wat bijkomende integratieproblemen introduceert. Ook wrapping en componentization worden in de literatuur genoemd als technieken om bestaande legacy-functionaliteit via moderne interfaces beschikbaar te maken (Comella-Dorda e.a., 2000). Ontwikkelaars worden geconfronteerd met methoden die niet voldoen aan de huidige standaarden voor testbaarheid of leesbaarheid, maar die vanwege hun kritieke rol in de berekeningen niet zomaar herschreven kunnen worden. Deze voorzichtigheid is herkenbaar in het werk van Feathers (2004), waar legacy-code benaderd wordt als code waarvan het bestaande gedrag eerst beschermd moet worden voordat ze veilig kan worden aangepast. Deze mix van oud en nieuw verhoogt de drempel voor nieuwe ontwikkelaars om de volledige flow van de applicatie te begrijpen.

6.4. Container-orchestratie en omgevingsbeheer

Tot slot vormt de integratie met Docker en Jupyter-notebooks een technische uitdaging. Zoals uitgebreid besproken in Hoofdstuk 11, start de applicatie op runtime containers op voor de onderzoekers. Dit vereist dat de webapplicatie directe controle heeft over de Docker-daemon. Docker waarschuwt in de eigen documentatie dat de daemon normaal met root-rechten werkt en dat toepassingen die via een API containers aanmaken daarom bijzonder voorzichtig moeten omgaan met invoer en parameters (Docker Inc., 2026c). Hierdoor is container-orchestratie in EMAV niet louter een implementatiedetail, maar een beveiligingskritisch onderdeel van de architectuur.

Daarnaast introduceert het netwerkmodel extra complexiteit. Containers zijn standaard geïsoleerd, maar het publiceren van poorten doorbreekt die isolatie door forwarding-regels naar de host te creëren (Docker Inc., 2026d). Wanneer bij port publishing geen expliciet IP-adres wordt opgegeven, kan Docker de poort bovendien standaard op alle interfaces publiceren (Docker Inc., 2026a). Voor een Jupyter-omgeving die via tokens toegankelijk is, betekent dit dat poortbindingen, bereik en tokenbeheer samen correct geconfigureerd moeten worden.

De noodzaak om DLL's dynamisch in te laden in een Linux-container, poortbindingen te beheren, resource-limieten in te stellen en tokens te extraheren uit logs, voegt een laag van systeem-engineering toe aan de applicatie. Ook resourcebeheer is hier relevant: Docker vermeldt dat containers zonder expliciete limieten standaard zoveel CPU en geheugen kunnen gebruiken als de kernel scheduler toelaat (Docker Inc., 2026e). Dit soort logica bevindt zich op het raakvlak tussen softwareontwikkeling en systeembeheer, wat het debugging-proces bemoeilijkt: een fout kan immers zowel in de C#-code, de Docker-configuratie als in de Python-omgeving van de container liggen.

Om deze uiteenlopende vormen van complexiteit — variërend van de wetenschappelijke detaillering tot de technische gelaagdheid van de data — beheersbaar te houden, is een ad-hoc programmeerstijl ontoreikend. In complexe softwarepro-

jecten is het net belangrijk om verantwoordelijkheden expliciet te scheiden en afhankelijkheden beheerst te organiseren; dit sluit aan bij de principes van Clean Architecture, waarbij de businesslogica beschermd wordt tegen details zoals frameworks, infrastructuur en externe technologieën (Martin, 2017). In plaats van losse, situatiegebonden oplossingen steunt de architectuur van EMAV-web daarom op een doordacht frame van software-ontwerppatronen (design patterns). Design patterns beschrijven volgens Gamma e.a. (1994) herbruikbare oplossingen voor terugkerende ontwerpproblemen in objectgeoriënteerde software. Deze patronen dienen dus niet louter als theoretische exercitie, maar vormen een direct antwoord op de hierboven geschetste uitdagingen.

Zo is het Repository patroon een noodzakelijk instrument om de hybride datastructuur te temmen, terwijl CQRS en het Mediator patroon de cognitieve last voor ontwikkelaars verlagen door verantwoordelijkheden strikt te scheiden. Tegelijkertijd biedt het Strategy patroon de nodige wetenschappelijke flexibiliteit binnen de rekenkern, en fungeert Dependency Injection als de lijm die al deze losgekoppelde componenten op een testbare manier samenbrengt. Deze keuze past binnen het bredere idee dat patronen en architectuurprincipes niet op zichzelf staan, maar gebruikt worden om domeinlogica onderhoudbaar, testbaar en technologisch minder afhankelijk te maken (Millett & Tune, 2015; Seemann & van Deursen, 2019). In de volgende subsecties worden de belangrijkste patronen in detail besproken, waarbij telkens de link wordt gelegd tussen het theoretische concept en de concrete meerwaarde voor de robuustheid en onderhoudbaarheid van EMAV.

6.5. Repository Pattern

Het repository patroon fungeert als een essentiële abstractielaag tussen de domeinlogica en de diverse gegevensbronnen waarover EMAV beschikt. Volgens Fowler (2003) gedraagt een repository zich als een in-memory verzameling van domeinobjecten, waarbij de technische details van de onderliggende persistentiemechanismen volledig worden geabstraheerd. In de context van EMAV is dit patroon cruciaal omdat het project opereert op een hybride datastructuur die over de jaren heen is geëvolueerd. Enerzijds is er een SQLite-database voor de opslag van ruwe inputgegevens en rekendata, en anderzijds een SQL Server-database voor de webgerelateerde metadata en gebruikersbeheer.

Binnen de codebase wordt dit gerealiseerd via een gelaagde hiërarchie van interfaces en implementaties. In het `ILV0.EMAV.Shared`-project bevindt zich de basisdefinitie `IBaseRepository<T>`, die algemene CRUD-operaties (Create, Read, Update, Delete) definieert. Specifieke projecten breiden dit uit, zoals `IWebBaseRepository<TEntity>` in de core-laag, waarbij gebruik wordt gemaakt van generieke constraints (where `TEntity` class, `IWebBaseEntity`). De `ILV0.EMAV.Core`-laag communiceert uitsluitend met deze interfaces. Hierdoor blijft de kern van de applicatie volledig onwetend over de specifieke implementatie details, of die nu gebruik-

maakt van Entity Framework Core in ILV0.EMAV.Infrastructure.Web of gespecialiseerde logic in de input-laag.

Deze abstractie verhoogt de onderhoudbaarheid aanzienlijk: indien een database-schema wijzigt of indien men besluit over te stappen naar een andere opslagtechnologie, hoeven de wijzigingen enkel in de infrastructuurlaag te worden doorgevoerd. De businesslogica blijft onaangetast. Bovendien is dit patroon de hoeksteen voor kwalitatieve unit testing. Door repositories te mocken, kan de businesslogica worden getoetst zonder afhankelijkheid van een actieve databaseverbinding. Dit wijst op een architecturale keuze die gericht is op stabiliteit op lange termijn en eenvoudige verifieerbaarheid van de code (Millett & Tune, 2015).

6.6. Command Query Responsibility Segregation (CQRS)

CQRS is een geavanceerd architectuurpatroon waarbij de verantwoordelijkheid voor het opvragen van gegevens (Queries) strikt wordt gescheiden van de verantwoordelijkheid voor het wijzigen van de systeemtoestand (Commands). Volgens Fowler (2011) biedt deze scheiding de mogelijkheid om elk pad onafhankelijk te structureren, te optimaliseren en te schalen. In een complex systeem zoals EMAV, waar berekeningen uren kunnen duren en metadata-opvragingen milliseconden moeten duren, is deze scheiding bittere noodzaak.

Binnen EMAV wordt CQRS geïmplementeerd met de LiteBus-bibliotheek (LiteBus Contributors, 2024). Dit resulteert in een zogenaamde Vertical Slice Architecture binnen de core-projecten (Bogard, 2018). Een actie zoals het verwijderen van een berekening wordt niet afgehandeld door een generieke CalculationService, maar door een specifieke DeleteCalculationCommand en de bijbehorende DeleteCalculationCommandHandler. Dit commando bevat enkel de minimale data (zoals een Guid Id) en de handler voert de noodzakelijke stappen uit: het ophalen van de metadata via een repository, het verwijderen van fysieke bestanden via een IFileManager en het finaal verwijderen van de database-records.

Aan de query-zijde worden specifieke modellen (DTO's) geretourneerd die geoptimaliseerd zijn voor de weergave in de UI, in plaats van de volledige domeinentiteiten te lekken. Deze aanpak voorkomt het ontstaan van Gouden Klassen die alle logica van een bepaald domein bevatten, in plaats van dat de logica gedistribueerd is over honderden kleine, gefocuste klassen die elk één specifieke taak hebben. Dit verlaagt de cognitieve last voor ontwikkelaars die aan het project werken aanzienlijk.

6.7. Mediator Pattern

Onlosmakelijk verbonden met de CQRS-aanpak is het mediator patroon. Dit patroon fungeert als een centrale verkeersleider die de communicatie tussen verschillende objecten coördineert, waardoor directe koppelingen tussen componenten

worden vermeden (Gamma e.a., 1994). In de moderne architectuur van EMAV is the IMediator (geleverd door LiteBus) het enige aanspreekpunt voor de presentatielaag (ILVO.EMAV.Web).

Een Razor Page in de UI-laag hoeft geen enkel inzicht te hebben in welke services, repositories of validators nodig zijn om een bepaalde actie uit te voeren. De pagina stuurt simpelweg een commando of query naar de mediator. De mediator zorgt er vervolgens voor dat dit bericht door de juiste pipelines passeert. Zo worden commando's automatisch eerst gevalideerd door IValidator-implementaties voordat ze de handler bereiken.

Dit zorgt voor een extreme vorm van ontkoppeling : de presentatielaag heeft geen enkele directe referentie naar de infrastructuur- of data-laag. Dit verhoogt de flexibiliteit van het systeem enorm; handlers kunnen worden herschreven, verplaatst naar andere microservices of gedecoreerd met extra logging zonder dat er ook maar één regel code in de UI-laag hoeft te wijzigen. Dit patroon is een directe indicatie van de architecturale volwassenheid van het project en draagt bij aan de robuustheid tegen technologische veranderingen.

6.8. Strategy Pattern

Het strategie patroon is een gedragspatroon dat toelaat om een algoritme te selecteren op runtime en dit algoritme in te kapselen in een uitwisselbaar object (Gamma e.a., 1994). In de rekenkern van EMAV is dit patroon van fundamenteel belang om de wetenschappelijke variabiliteit van de emissiemodellen te ondersteunen. De kerninterface ICalculator<Tin, Tout> vormt de basis for een uitgebreide familie van rekenstrategieën.

Deze architectuur is essentieel omdat EMAV berekeningen uitvoert voor diverse domeinen, zoals enterische emissies, stalemissies en mestverwerking. Elke berekening heeft haar eigen set formules, inputparameters en validatieregels. Door elke berekening als een afzonderlijke strategie (bijvoorbeeld BerekenEmissieLandbouwbodem) te implementeren, blijft de overkoepelende orchestratielogica clean en overzichtelijk. De orchestrator roept simpelweg de DoCalc-methode aan op de geïnjecteerde strategie.

Dit bevordert het Open-Closed Principle (onderdeel van SOLID): de rekenkern staat open voor uitbreiding (men kan eenvoudig een nieuwe reken module toevoegen door de interface te implementeren), maar is gesloten voor aanpassingen (de bestaande orchestratie hoeft niet gewijzigd te worden). Voor een project waar wetenschappers regelmatig nieuwe formules aanleveren, is dit een cruciale eigenschap die voorkomt dat het project na verloop van tijd vastloopt in haar eigen complexiteit.

6.9. Factory Pattern

Het factory patroon wordt aangewend om de creatie van objecten te abstraheren van de client-code. Dit is met name nuttig wanneer het exacte type van het aan te maken object afhankelijk is van configuratie of gebruikersinput op runtime. In EMAV wordt dit patroon prominent toegepast in de `IExporterFactory`.

Het systeem ondersteunt een breed scala aan exportmogelijkheden, waaronder CSV voor verdere analyse in statistische software, Excel voor rapportage en JSON voor integratie met andere systemen. De `ExporterFactory` neemt een `Export-Type` als parameter en retourneert de juiste implementatie van de `IExporter`-interface. Hierdoor hoeft de aanroepende code niet te weten hoe een specifieke exporter geconfigureerd moet worden.

Deze centralisatie van creatielogica voorkomt codeduplicatie en maakt het systeem zeer modulair. Indien er in de toekomst ondersteuning voor een nieuw formaat moet worden toegevoegd, hoeft enkel een nieuwe klasse te worden geschreven en geregistreerd in de factory. De rest van de applicatie blijft volledig ongewijzigd. Dit illustreert de visie van het project op uitbreidbaarheid en het vermijden van harde koppelingen tussen componenten (Gamma e.a., 1994).

6.10. Facade Pattern

Gezien de enorme interne complexiteit van de EMAV-rekenkern, waarbij tientallen subsystemen gelijktijdig actief zijn, is het facade patroon onontbeerlijk. Een facade biedt een vereenvoudigde interface naar een complex subsysteem, waardoor de interne complexiteit wordt afgeschermd voor de gebruiker (Gamma e.a., 1994). In EMAV fungeren de belangrijkste controllers en orkestratie-services als facades.

Wanneer een gebruiker op Start Berekening klikt, lijkt dit voor de UI een enkele actie. Achter de facade vindt echter een complexe dans plaats: inputbestanden worden gevalideerd, de juiste rekenfactoren worden uit de database opgehaald, cache-mechanismen worden geraadpleegd, meerdere berekeningsstrategieën worden in de juiste volgorde aangeroepen en de resultaten worden finaal geaggregeerd en weggeschreven. De facade verbergt deze interacties en biedt een stabiele API. Dit is een erkenning van de inherente complexiteit van het domein: in plaats van te proberen de complexiteit volledig weg te nemen, wordt ze beheersbaar gemaakt door ze te groeperen achter logische, gebruiksvriendelijke interfaces. Dit verbetert de onderhoudbaarheid omdat de interne werking van de rekenkern kan worden geoptimaliseerd zonder dat de UI-code hiervan hinder ondervindt.

6.11. Dependency Injection en Inversion of Control

Dependency Injection is de architecturale ruggengraat die alle voorgaande patronen in EMAV faciliteert en verbindt. In plaats van dat klassen hun eigen afhankelijkheden instantiëren, worden deze afhankelijkheden van buitenaf aangeboden.

Volgens Seemann en van Deursen (2019) is DI de enige manier om een echt modulaire en testbare software-architectuur te bouwen.

EMAV maakt optimaal gebruik van de ingebouwde DI-container van .NET. Elk deelproject (Core, Infrastructure, Web) bevat een eigen `DependencyInjection.cs`-bestand waarin de interne services, handlers en repositories worden geregistreerd. Dit bevordert een hoge mate van inkapseling: de UI-laag hoeft enkel de DI-methode van de Core-laag aan te roepen om alle benodigde logica beschikbaar te maken.

Dit mechanisme is cruciaal voor de flexibiliteit van het project. Het laat toe om op een transparante manier implementaties te wisselen op basis van de omgeving of voor testdoeleinden. DI is de motor achter de andere patronen; zonder een goede DI-structuur zouden patronen zoals Strategy en Repository leiden tot een onhoudbare hoeveelheid handmatige object-creatie doorheen de applicatie.

6.12. Adapter Pattern

Het adapter patroon maakt het mogelijk dat klassen met incompatibele interfaces toch naadloos kunnen samenwerken (Gamma e.a., 1994). In de context van EMAV is dit patroon een cruciaal instrument om de kloof tussen de moderne Clean Architecture en de waardevolle legacy-code overbruggen. Zoals eerder beschreven, bevat het project logica die overgenomen is uit een oudere Windows Forms-omgeving. In plaats van deze oude code integraal te herschrijven, worden adapters ingezet. Deze adapters presenteren de oude functionaliteit aan de nieuwe applicatie via een interface die voldoet aan de nieuwe standaarden (bijvoorbeeld door gebruik te maken van `Task<T>` voor asynchroon werken). Hierdoor kan de nieuwe kernlogica modern en clean blijven, terwijl ze toch gebruikmaakt van de bewezen nauwkeurigheid van de bestaande rekenmodules. Dit toont een pragmatische en architecturaal verantwoorde aanpak van software-evolutie aan.

6.13. Decorator Pattern en Result Pattern

Het decorator patroon voegt dynamisch verantwoordelijkheden toe aan een object zonder de onderliggende klasse te wijzigen (Gamma e.a., 1994). In de EMAV-codebase wordt dit principe toegepast via de mediator pipeline. Elke actie die door het systeem stroomt, wordt gedecoreerd met extra functionaliteit zoals automatische validatie via `FluentValidation`. Hierdoor blijven de eigenlijke business-handlers puur en gefocust op hun specifieke taak.

Daarnaast wordt het Result Pattern (via de `ErrorOr` bibliotheek) consistent toegepast. In plaats van uitzonderingen (exceptions) te gebruiken voor control flow bij te verwachten fouten, retourneren methoden een object dat ofwel het resultaat, ofwel een lijst van fouten bevat. Dit patroon, zoals beschreven door Khorikov (2020), zorgt voor een zeer voorspelbare en robuuste foutafhandeling die naadloos inte-

greert met de Clean Architecture-filosofie en functionele programmeerprincipes.

6.14. Clean Architecture

Alle voorgaande patronen komen samen in de overkoepelende Clean Architecture-structuur van EMAV. Volgens Martin (2017) is het hoofddoel van deze architectuur om de businesslogica volledig te isoleren van infrastructuur, frameworks en externe invloeden. In de projectstructuur van EMAV is deze filosofie strikt doorgevoerd:

- Domain Layer (ILVO.EMAV.Data.*): Bevat de pure entiteiten en domeinobjecten.
- Application Layer (ILVO.EMAV.Core): Bevat de businessregels, de mediator handlers en de definities van de interfaces.
- Infrastructure Layer (ILVO.EMAV.Infrastructure.*): Bevat de technische implementaties, zoals database-connectiviteit en bestandssysteem-interacties.
- Presentation Layer (ILVO.EMAV.Web): De user interface (Razor Pages) die uitsluitend communiceert met de Core-laag.

Deze gelaagdheid zorgt ervoor dat EMAV een zeer hoge mate van technologische onafhankelijkheid geniet. De Clean Architecture is de ultieme uiting van de complexiteitsbeheersing binnen dit project: het biedt een duurzaam frame waarin alle andere design patterns optimaal tot hun recht komen om samen een kwalitatief superieur softwareproduct te vormen.

6.15. Besluit: Een synergetische oplossing voor gelaagde complexiteit

De diepgaande analyse van de diverse complexiteitsfactoren binnen EMAV-web — van de wetenschappelijke domeinlogica tot de infrastructurele orkestratie — toont aan dat een enkelvoudige technische ingreep onvoldoende zou zijn. Elk van de besproken design patterns vormt een essentiële stap om een specifieke barrière te doorbreken: de *Repositories* temmen de hybride datastructuur, *CQRS* en de *Mediator* verlagen de cognitieve last, *Strategieën* bieden wetenschappelijke flexibiliteit en *Adapters* slaan de brug naar de waardevolle legacy-kern.

Het is echter de bewuste synergie tussen deze patronen en de overkoepelende *Clean Architecture* die de werkelijke oplossing vormt. Door de businesslogica strikt te isoleren via deze patronen, wordt een systeem gecreëerd dat niet alleen technisch robuust is voor IT-ontwikkelaars, maar dat ook de noodzakelijke transparantie biedt om wetenschappelijke modellen voor onderzoekers te ontsluiten. Deze gelaagde architecturale aanpak is de meest solide en duurzame keuze voor het

project; het vormt het onmisbare fundament voor de hybride integratiestrategie die in de volgende hoofdstukken wordt gevalideerd via de Proof of Concept.

7

Tekortschiëting van documentatie

De complexiteit van EMAV-web, zoals geanalyseerd in de voorgaande hoofdstukken, stelt hoge eisen aan de wijze waarop de werking van het systeem wordt vastgelegd. Hoewel er binnen ILVO diverse pogingen zijn ondernomen om zowel de technische als de wetenschappelijke aspecten van de software te documenteren, blijkt in de praktijk dat deze traditionele methoden fundamenteel tekortschiëten. In dit hoofdstuk wordt dieper ingegaan op de oorzaken van dit falen, waarbij de focus ligt op de structurele barrières die statische documentatie opwerpt voor een effectieve samenwerking tussen IT en onderzoek.

7.1. Het probleem van "Documentation Drift"

Een van de meest hardnekkige problemen binnen het EMAV-project is de zogenaamde documentation drift: het fenomeen waarbij de geschreven documentatie en de feitelijke broncode na verloop van tijd uit elkaar groeien. Dit probleem is niet uniek voor EMAV. Tan e.a. (2024) tonen aan dat softwaredocumentatie verouderd kan raken wanneer verwijzingen naar code-elementen in de documentatie blijven bestaan nadat die elementen in de broncode gewijzigd of verwijderd zijn. Omdat de emissiemodellen regelmatig worden bijgewerkt op basis van nieuwe wetenschappelijke inzichten, moet de documentatie telkens handmatig worden gesynchroniseerd. In een omgeving met hoge tijdsdruk wordt dit vaak verwaarloosd, waardoor onderzoekers beslissingen baseren op verouderde informatie. Binnen de C#-codebase uit zich dit vaak in commentaren die niet langer overeenkomen met de berekeningslogica in de handlers of rekenmodules. De risico's hiervan zijn ook empirisch onderzocht: Ibrahim e.a. (2012) beschrijven dat verouderde comments ontwikkelaars op het verkeerde spoor kunnen zetten en dat patronen in

commentaarupdates voorspellende waarde hebben voor toekomstige bugs. Wanneer een onderzoeker vraagt "hoe wordt parameter X berekend?", moet een programmeur vaak de code induiken omdat de externe documentatie (zoals Word-bestanden of Wiki-pagina's) niet langer betrouwbaar is. Dit proces is inefficiënt en verhoogt de kans op foutieve interpretaties van de wetenschappelijke modellen.

7.2. Technische focus versus domeinfocus

Bestaande automatische documentatietools voor .NET, zoals DocFX of Swagger, zijn primair ontworpen voor softwareontwikkelaars. DocFX positioneert zich als een generator voor .NET API-documentatie op basis van broncode en XML-commentaren (.NET Foundation, 2025). Swagger/OpenAPI beschrijft op zijn beurt API's in termen van paths, operaties, parameters, request bodies en responses (SmartBear Software, 2026). Deze tools excelleren daardoor in het weergeven van de technische interface (API-endpoints, klassenhiërarchieën, methodesiganturen), maar falen in het uitleggen van de intentie en de wiskundige onderbouwing van de logica. Dit sluit aan bij onderzoek naar API-documentatie: Meng e.a. (2018) benadrukken dat API-documentatie vooral moet beantwoorden aan de informatiebehoeften van softwareontwikkelaars.

Voor een onderzoeker is de informatie dat een methode `CalculateEmission(double factor)` heet en een `Task<double>` retourneert, van weinig waarde. Wat de onderzoeker nodig heeft, is inzicht in welke wetenschappelijke bronnen aan de basis liggen van de gebruikte coëfficiënten en hoe de formules onderling samenhangen. Vanuit Domain-Driven Design wordt dit spanningsveld herkend als een taal- en modelprobleem: de software moet een gedeelde domeintaal ondersteunen die door ontwikkelaars en domeinexperts begrepen wordt (Khononov, 2022). De huidige documentatievormen zijn te "IT-centrisch" en bieden geen abstractieniveau dat aansluit bij de leefwereld van de wetenschapper.

7.3. Gebrek aan interactiviteit en executabiliteit

Traditionele documentatie is statisch. Een onderzoeker kan een PDF-document wel lezen, maar hij kan er niet mee experimenteren. In een project als EMAN is het essentieel dat onderzoekers de impact van parameterwijzigingen direct kunnen observeren. Statische documentatie dwingt de onderzoeker in een passieve rol: hij moet de logica in de tekst begrijpen, deze mentaal vertalen naar de code, en vervolgens wachten op de IT-afdeling om een aanpassing door te voeren.

Het ontbreken van een executabel aspect in de documentatie betekent dat er geen levendelink is tussen de uitleg en de uitvoering. Dit gebrek aan interactiviteit versterkt het black-box effect van de applicatie. De onderzoekers hebben nood aan een vorm van documentatie die niet alleen beschrijft wat de code doet, maar die hen ook in staat stelt om de logica te valideren door middel van directe interactie,

zonder dat zij daarvoor de volledige ontwikkelomgeving hoeven te beheersen.

7.4. Besluit: De barrière van statische kennisoverdracht

De inventarisatie van de tekortkomingen van de huidige documentatie leidt tot drie cruciale inzichten. Ten eerste is er het onvermijdelijke fenomeen van *documentation drift*, waardoor statische documenten in een dynamische onderzoeksomgeving snel hun betrouwbaarheid verliezen. Ten tweede is er een fundamentele mismatch tussen de informatiebehoefte van onderzoekers en de output van bestaande .NET-documentatietools, die te eenzijdig gericht zijn op de technische implementatie in plaats van de wetenschappelijke intentie. Ten slotte zorgt het gebrek aan executabiliteit ervoor dat documentatie een passief naslagwerk blijft, in plaats van een actief instrument voor validatie.

De belangrijkste les uit dit hoofdstuk is dat de huidige documentatievormen de “black-box” beleving van de onderzoekers niet wegnemen, maar juist bestendigen. Het louter beschrijven van code is onvoldoende om de kloof tussen IT en wetenschap te dichten. Dit inzicht vormt de noodzakelijke motivatie om te zoeken naar alternatieve documentatievormen die tekst en executie onlosmakelijk met elkaar verbinden, een onderwerp dat in het volgende hoofdstuk centraal staat.

8

Alternatieve vormen van documentatie

Om de tekortkomingen van traditionele documentatie te overbruggen, wordt er in dit onderzoek gekeken naar alternatieve vormen die de scheiding tussen uitleg en uitvoering opheffen. De meest veelbelovende benaderingen zijn geworteld in de filosofie van Literate Programming en de moderne praktische uitwerking daarvan in de vorm van computationele notebooks. Deze vormen transformeren documentatie van een statisch bijproduct naar een actief, uitvoerbaar en centraal instrument binnen de wetenschappelijke workflow.

8.1. Het fundament: Literate Programming

Zoals geïntroduceerd door Knuth (1992), draait Literate Programming om de veronderstelling dat een computerprogramma in de eerste plaats geschreven moet worden als een narratief voor menselijke lezers, waarbij de machine-uitvoerbare code een tweede bijproduct is. In plaats van tekstuele uitleg als secundaire commentaar toe te voegen aan de code (code with comments), vormt bij Literate Programming de tekst het primaire raamwerk waarin de codefragmenten zijn ingebed.

De essentie van Literate Programming ligt in de twee fundamentele operaties: *tangle* en *weave*.

- **Tangle:** Deze operatie extraheert de broncode uit het bronbestand en ordent deze in een formaat dat door de compiler of interpreter kan worden uitgevoerd. Voor EMANV betekent dit dat de wetenschappelijke formules die in een narratief document zijn beschreven, direct worden omgezet naar de C#-logica die de berekeningen aandrijft. Dit garandeert dat de logica zoals gedocumenteerd identiek is aan de logica zoals geïmplementeerd.

- **Weave:** Deze operatie genereert de leesbare documentatie (zoals een PDF of HTML-pagina) uit hetzelfde bronbestand. Hierbij worden de codefragmenten gepresenteerd in hun context, vaak vergezeld van rijke visualisaties, wiskundige notaties in $\text{\LaTeX}$ en uitgebreide kruisverwijzingen.

Voor het EMAV-project biedt dit perspectief een structurele oplossing voor de documentation drift. Wanneer de wetenschappelijke verantwoording en de eigenlijke berekeningslogica in hetzelfde artefact zijn ondergebracht, verdwijnt de noodzaak voor handmatige synchronisatie. Het document fungeert als een "Executable Paper", waarbij elke bewering over het model direct kan worden gestaafd door de bijbehorende code uit te voeren.

8.2. Computationale notebook-platforms

Computationale notebooks zijn de moderne, interactieve erfgenamen van Knuths visie. Ze bieden een REPL-achtige (Read-Eval-Print Loop) ervaring die ideaal is voor exploratief onderzoek. Voor de implementatie binnen ILVO zijn drie belangrijke platforms overwogen, elk met hun eigen sterktes en zwaktes binnen een .NET-georiënteerde omgeving.

8.2.1. Jupyter-notebooks

Jupyter (Project Jupyter, [2025](#)) is de de facto standaard in de data science wereld. Het blinkt uit door zijn enorme ecosysteem en de ondersteuning voor honderden "kernels". In dit project is gekozen voor Jupyter vanwege de volwassenheid van de webinterface en de uitstekende integratie met Docker-omgevingen. Een specifiek voordeel voor EMAV is de mogelijkheid om via de Python.NET bridge de C#-kern direct aan te spreken vanuit een Python-kernel. Dit laat onderzoekers toe om Python's krachtige visualisatie-libraries (zoals Matplotlib of Seaborn) te gebruiken op data die is gegenereerd door de robuuste .NET-berekeningsengine. Het grootste nadeel van Jupyter is de niet-lineaire uitvoervolgorde van cellen, wat discipline vereist van de onderzoeker om de reproduceerbaarheid te waarborgen (Rule e.a., [2018](#)).

8.2.2. Quarto

Quarto (Posit Software, PBC, [g.d.](#)) is een technisch meer geavanceerde opvolger die specifiek focust op wetenschappelijke publicaties en technische rapportage. Het grote voordeel van Quarto is dat het gebruikmaakt van platte tekstbestanden (Markdown), wat het versiebeheer via Git aanzienlijk vergemakkelijkt in vergelijking met de JSON-gebaseerde .ipynb bestanden van Jupyter. Quarto ondersteunt ook .multi-languagedocumenten, waarbij verschillende talen in één flow kunnen worden gecombineerd. Hoewel Quarto een sterke kandidaat is voor de finale publicatie van wetenschappelijke rapporten binnen ILVO, is de interactieve ervaring voor

de gemiddelde onderzoeker momenteel nog minder intuïtief dan bij de klassieke Jupyter interface.

8.2.3. Org-mode & babel

Org-mode (Dominik & Schulte, [2025](#)) biedt wellicht de meest pure vorm van Literate Programming binnen de Emacs-omgeving. Met de Babel-extensie kunnen codeblokken in nagenoeg elke taal worden uitgevoerd en kunnen resultaten direct in het document worden teruggekoppeld. Voor de doeleinden van EMAV is Org-mode echter minder geschikt als breed verspreide oplossing. De steile leercurve van Emacs vormt een te grote barrière voor de gemiddelde wetenschapper die snel een berekening wil valideren. Het blijft echter een krachtige tool voor power users en ontwikkelaars die een maximale controle over hun documentatieproces wensen.

8.3. De Techniek achter de Verweving: C# Interop in Notebooks

Een cruciaal aspect van de gekozen documentatievorm is de manier waarop de complexe .NET-codebase toegankelijk wordt gemaakt in de interactieve notebook-omgeving. In plaats van de logica te herimplementeren in een scripttaal (wat de betrouwbaarheid zou ondermijnen), wordt de eigenlijke ILVO.EMAV.Core.dll dynamisch ingeladen.

Dit proces verloopt via een startup-script dat gebruikmaakt van de Common Language Runtime (CLR). Hierdoor kunnen onderzoekers in hun notebook direct objecten instantiëren van C#-klassen, methoden aanroepen en zelfs LINQ-queries uitvoeren op de data-modellen. Deze aanpak garandeert dat de documentatie altijd werkt op de exacte versie van de code die ook in de productieomgeving van EMAV-web draait. Het opheffen van de technische barrière tussen de gecompileerde C#-wereld en de interactieve data science-wereld is de belangrijkste stap in het verkleinen van de kloof tussen IT en wetenschap.

8.4. Interactieve documentatie als leer- en validatieplatform

De integratie van notebooks transformeert de documentatie van een passief naslagwerk naar een actieve leeromgeving. Naast de pure berekeningslogica kunnen notebooks worden verrijkt met interactieve elementen zoals sliders voor parameter-exploratie, dynamische grafieken en directe links naar externe wetenschappelijke bronnen.

Dit zorgt voor een gelaagde informatievoorziening die aansluit bij verschillende behoeften:

1. **De Narratieve Laag:** De wetenschappelijke tekst die de context, de wiskundige afleidingen en de verantwoording van het model biedt.
2. **De Executabele Laag:** De levende codefragmenten die de formules direct implementeren en uitvoerbaar maken.
3. **De Visualisatielaag:** Grafieken en tabellen die de output van de code direct inzichtelijk maken en vergelijking met historische data mogelijk maken.

Door deze drie lagen in één document te verenigen, wordt voldaan aan de behoefte van de onderzoeker aan zowel transparantie als autonomie. Het resultaat is een documentatievorm die niet alleen beschrijft hoe het systeem werkt, maar die zelf een integraal en onmisbaar onderdeel vormt van het kwaliteitsborgingsproces van het instituut.

8.5. Besluit: Evaluatie van de alternatieven

De verkenning van alternatieve documentatievormen toont aan dat er diverse platformen beschikbaar zijn die de principes van Literate Programming vertalen naar de moderne onderzoekspraktijk. Uit de analyse van de vier belangrijkste alternatieven kunnen de volgende conclusies worden getrokken:

- **Jupyter-notebooks:** Blinkt uit door een enorm ecosysteem en naadloze integratie met Docker, wat essentieel is voor een veilige PoC. Het grootste nadeel is de niet-lineaire uitvoervolgorde, die discipline vereist voor reproduceerbaarheid.
- **Quarto:** Biedt superieur versiebeheer door het gebruik van platte tekstbestanden (Markdown) en is ideaal voor publicaties. Echter, de interactieve ervaring is momenteel minder toegankelijk voor onderzoekers die gewend zijn aan een directe notebook-interface.
- **Org-mode & Babel:** De meest flexibele en pure vorm van Literate Programming, maar de zeer steile leercurve van Emacs maakt het ongeschikt als brede oplossing binnen een institutionele context zoals ILVO.
- **RMarkdown:** Een bewezen standaard voor dynamische rapportage, maar de sterke focus op de R-taal maakt het minder optimaal voor een omgeving waar de integratie met .NET via Python centraal staat.

Op basis van deze vergelijking is Jupyter-notebooks geselecteerd als het primaire platform voor de Proof of Concept. De combinatie van gebruiksvriendelijkheid, de krachtige Python.NET bridge en de containerisatie weegt zwaarder dan de nadelen van de andere formaten. Hiermee is de keuze voor de interactieve laag van de architectuur gemaakt, die in de volgende hoofdstukken technisch zal worden onderbouwd.

9

Veilig experimenteren

Een van de belangrijkste doelstellingen van dit onderzoek is om onderzoekers de vrijheid te geven om te experimenteren met modellen, zonder dat dit een risico vormt voor de stabiliteit van de productieomgeving. Veilig experimenteren vereist zowel technische isolatie op procesniveau als functionele vangrails op dataniveau.

9.1. Technische isolatie via Docker-sandboxing

Zoals gedetailleerd beschreven in de technische opzet (zie Hoofdstuk 11), vormt containerisatie via Docker de infrastructurele basis voor veiligheid (Boettiger, 2015). Elke onderzoeker krijgt bij het opstarten van een sessie een eigen, kortstondige (ephemeral) Jupyter-container (Jupyter e.a., 2018).

Dit sandboxing-mechanisme biedt bescherming op meerdere niveaus (Bui, 2015; Zheng & Wenhao, 2020):

Bestandssysteem: De container heeft een eigen read-only root-bestandssysteem via Linux namespaces. Wijzigingen die een onderzoeker aanbrengt aan de systeemconfiguratie van de container gaan verloren na het afsluiten van de sessie, waardoor de omgeving altijd schoon blijft voor de volgende run.

Resource-limieten: Door gebruik te maken van Docker's resource-quota (CPU en geheugen via control groups) kan worden voorkomen dat een experimenteel script de volledige server platlegt. Mocht een onderzoeker per ongeluk een geheugenlek of een oneindige lus introduceren, dan zal enkel zijn individuele container worden gepauzeerd of getermineerd.

Netwerk-isolatie: De Jupyter-container bevindt zich in een afgeschermd netwerk-segment. Hij kan enkel communiceren met de noodzakelijke databronnen en de web-API van EMAV, maar heeft geen ongecontroleerde toegang tot het bredere netwerk van ILVO.

Deze isolatie verlaagt de psychologische drempel voor onderzoekers om “wat-als”-scenario’s te verkennen. De angst om het “echte” systeem te breken wordt technisch weggenomen door de sandbox-architectuur.

9.2. Data-integriteit en lokale data-cloning

Veilig experimenteren betekent ook dat de onderzoeker moet kunnen werken met realistische data zonder de centrale metadata-database te vervuilen (Johnson & Fotouhi, 1996). Binnen de EMANotebooks wordt gebruikgemaakt van een strategie van local cloning.

Wanneer een onderzoeker een analyse start, wordt er een tijdelijke kopie gemaakt van de relevante SQLite-bestanden met inputgegevens. De onderzoeker kan parameters in deze lokale kopieën naar hartenlust aanpassen, verwijderen of toevoegen om de gevoeligheid van het model te testen. De officiële database blijft hierbij onaangetast. Door de resultaten van een experimentele run direct in de notebook te vergelijken met de resultaten van het officiële model, krijgt de onderzoeker onmiddellijk visuele feedback (Kohavi e.a., 2020). Dit proces van directe validatie fungeert als een functionele vangrail: fouten in de wetenschappelijke logica worden direct zichtbaar in de grafieken, nog voordat er sprake is van een formele aanvraag voor een codewijziging.

9.3. De weg van experiment naar productie: gecontroleerde promotie

Tot slot maakt de voorgestelde omgeving een cruciaal onderscheid tussen een “vrij experiment” en een “formele model-update”. Een notebook dient als de zandbak waarin een nieuwe wetenschappelijke formule wordt verfijnd en gevalideerd over meerdere iteraties (Tamburri, 2020).

Zodra een onderzoeker overtuigd is van de meerwaarde van een wijziging, fungeert de notebook als de technische specificatie voor de IT-afdeling. In plaats van een vaag tekstdocument, overhandigt de onderzoeker een werkend code-exemplaar. De IT-ontwikkelaar neemt vervolgens de taak op zich om deze gevalideerde logica structureel te integreren in de C#-rekenmodules, inclusief de bijbehorende unit tests en architecturale optimalisaties (Bosch e.a., 2021). Dit schept een heldere verdeling van verantwoordelijkheden: de onderzoeker is de eigenaar van de wetenschappelijke innovatie, terwijl de IT-afdeling de bewaker blijft van de softwarekwaliteit en stabiliteit.

9.4. Besluit: Vangrails voor innovatie

De analyse van de noodzakelijke randvoorwaarden voor veilig experimenteren leidt tot de conclusie dat een hybride omgeving alleen succesvol kan zijn wanneer er

strikte “vangrails” aanwezig zijn. De drie besproken pijlers vormen hierbij een onlosmakelijk geheel. De technische isolatie via Docker-sandboxen garandeert dat de infrastructuur beschermd blijft tegen onbedoelde overbelasting. De strategie van lokale data-clonering zorgt ervoor dat de wetenschappelijke integriteit van de centrale data nooit in het gedrang komt. Ten slotte biedt het promotieproces van notebook naar productie een heldere workflow die de samenwerking tussen IT en onderzoek professionaliseert.

Door deze maatregelen verschuift de beleving van de onderzoeker van angst om het systeem te beschadigen naar de vrijheid om autonoom hypothesen te toetsen. Dit vormt de essentiële basis voor de nauwere verweving tussen code en documentatie, waarbij de notebook niet langer slechts een passief document is, maar een veilige en uitvoerbare extensie van de EMAN-codebase.

10

Nauwere verweving tussen code & documentatie

De uiteindelijke meerwaarde van de voorgestelde aanpak ligt in de fundamentele verschuiving van documentatie als een statisch artefact naar documentatie als een integraal onderdeel van de software-levenscyclus. Door de scheiding tussen deze twee werelden op te heffen, ontstaat een ecosysteem dat zowel wetenschappelijk accuraat als technisch robuust is.

10.1. Minder communicatie-overhead en ruis

Een van de meest tastbare voordelen van de nauwere verweving is de drastische daling in de communicatie-overhead. In de traditionele workflow ging veel tijd verloren aan het verduidelijken van specificaties. Onderzoekers moesten wachten op IT om te zien hoe een formule in de praktijk presteerde, waarna IT vaak moest terugkomen bij de onderzoekers omdat de wiskundige beschrijving in het Word-document ambigu was.

In de nieuwe workflow fungeert de Jupyter-notebook als een gemeenschappelijke taal (ubiquitous language). De programmeur ziet de C#-interop en de achterliggende types, terwijl de onderzoeker de resultaten en de narratieve tekst ziet. Dit gedeelde referentiekader minimaliseert de kans op spraakverwarring. Vragen over de interne werking van de code kunnen nu vaak in enkele minuten door de onderzoeker zelf worden beantwoord door simpelweg een code-cel in de Jupyter-notebook uit te voeren.

10.2. Documentatie als "Single Source of Truth"

Door de principes van Literate Programming te omarmen, wordt documentatie niet langer een sluitpost aan het einde van een project, maar de drijvende kracht

erachter. Elke wijziging in een wetenschappelijk model wordt gelijktijdig gedocumenteerd (in tekst) en geïmplementeerd (in code) binnen hetzelfde proces. Dit verhoogt de kwaliteit van het intellectueel eigendom van ILVO aanzienlijk. De kennis zit niet meer gefragmenteerd in de hoofden van individuele experts of in een wirwar van ongecontroleerde scripts op lokale computers. In plaats daarvan is er één traceerbare bron waar de wetenschappelijke redenering en de technische executie samenkomen. Dit is essentieel voor de reproduceerbaarheid van het onderzoek, aangezien men jaren later nog exact kan zien welke aannames tot welke codewijzigingen hebben geleid.

10.3. Conclusie: een strategische transformatie

De integratie van geavanceerde software-architectuur, container-isolatie en interactieve documentatievormen transformeert de relatie tussen IT en onderzoek fundamenteel. De IT-afdeling verschuift van een uitvoerende partij naar een platform-enabler. Zij faciliteren de infrastructuur en de robuuste backend, terwijl de onderzoekers de domeinlogica voeden, verfijnen en valideren met een voorheen ongekende mate van autonomie.

Deze nauwe verweving vormt een veelbelovende oplossingsrichting voor de complexiteitsuitdagingen binnen EMAN-web. Het onderzoek en de bijbehorende proof-of-concept tonen aan dat een platform waarop technologische vernieuwing en wetenschappelijke precisie elkaar versterken, technisch haalbaar is. Door de kloof tussen code en documentatie te verkleinen, wordt een fundament gelegd dat de potentie heeft om de wendbaarheid en de kwaliteit van het wetenschappelijk onderzoek bij ILVO op lange termijn structureel te ondersteunen. De succesvolle validatie van dit concept opent de deur naar een meer geïntegreerde werkwijze waarbij IT en wetenschap elkaar optimaal versterken.

11

Proof of Concept: hybride implementatiestrategie

Om de theoretische concepten rond onderzoeksautonomie en de nauwe verweving van code en documentatie te valideren, is een geïntegreerd systeem ontwikkeld als Proof of Concept (PoC). Dit systeem fungeert als een georkestreerd ecosysteem waarbij de robuuste .NET-backend van ILVO de regie voert over interactieve Jupyter-omgevingen. De kern van de oplossing ligt in een hybride, 'switchable' architectuur: een opzet die het mogelijk maakt om op granulair niveau te wisselen tussen de officieel gecompileerde C#-implementatie en een interpretatieve, door de onderzoeker aangestuurde Python-omgeving. Hiermee wordt een dynamische samenwerkingsvorm gerealiseerd waarbij de stabiliteit van de backend gewaarborgd blijft, terwijl de onderzoeker de vrijheid krijgt om direct op de actuele rekenmodellen te experimenteren. Dit hoofdstuk biedt een diepgaand overzicht van de infrastructuur, de architecturale keuzes en de technische realisatie van deze proefopstelling.

11.1. Infrastructuur: docker-orchestratie

De PoC rust op een volledig geautomatiseerde infrastructuur die ervoor zorgt dat elke onderzoeker over een identieke, geïsoleerde en reproduceerbare analyseomgeving beschikt. Het handmatig opzetten van dergelijke omgevingen door onderzoekers zelf zou leiden tot versieconflicten en configuratiefouten. Daarom is gekozen voor orchestratie direct vanuit de C#-backend van de applicatie.

11.1.1. Geautomatiseerde DLL-collectie en versiebeheer

Een fundamentele uitdaging bij het werken met hybride talen is het garanderen dat de Python-scripts opereren op exact dezelfde versie van de domeinlogica als

de hoofdapplicatie. Binnen de `JupyterSetup`-klasse is een robuust mechanisme geïmplementeerd voor DLL-collectie.

De methode `SetupDLL` berekent eerst de root van de solution op basis van de locatie van de uitvoerende assembly (`Assembly.GetExecutingAssembly().Location`). Vervolgens doorzoekt het mechanisme alle `obj`-directories. De complexiteit hierbij ligt in het filteren van de juiste bestanden: enkel de effectief gecompileerde bibliotheken van de EMAN-kern en data-projecten worden geselecteerd. Referentie-assemblies, debug-symbolen en platformspecifieke runtimes worden expliciet uitgesloten. Door dit proces te automatiseren, is de analyseomgeving altijd gesynchroniseerd met de laatst gebouwde versie van de applicatie, wat de "Single Source of Truth" op binair niveau waarborgt.

11.1.2. Programmatisch beheer via Docker.DotNet

Voor de eigenlijke orchestratie van de containers wordt gebruikgemaakt van de bibliotheek `Docker.DotNet`. Deze keuze laat toe om de volledige levenscyclus van een analyseomgeving te beheren zonder dat de gebruiker de command-line interface van Docker hoeft te kennen.

Een cruciaal onderdeel van de implementatie is de platformonafhankelijke communicatielaag. De applicatie detecteert op welk besturingssysteem zij draait en past de verbindingsmethode hierop aan. Op Windows-ontwikkelmachines wordt gebruikgemaakt van named pipes (`npipe:///pipe/docker_engine`), terwijl op de Linux-gebaseerde productieservers van ILVO rechtstreeks via Unix-sockets (`unix:///var/run/docker.sock`) wordt gecommuniceerd. De backend zorgt niet alleen voor het aanmaken van de container, maar configureert ook de noodzakelijke netwerk-isolatie en resource-limieten.

11.1.3. Log-parsing en dynamische token-extractie

Wanneer een Jupyter-container opstart, genereert deze automatisch een unieke beveiligingstoken voor authenticatie. Om de ervaring voor de onderzoeker zo drempelloos mogelijk te maken, wordt deze token automatisch geëxtraheerd. De C#-backend monitort de log-stream van de container.

Aangezien de Docker-daemon de logs in een gemultiplexed binair formaat verstuurt, is een gespecialiseerde `DemultiplexStream`-methode geïmplementeerd om dit te vertalen naar leesbare tekst. Met behulp van de reguliere expressie `token=([a-f0-9]+)` wordt de token geïsoleerd. Samen met de dynamisch door Docker toegewezen poort vormt dit een unieke URL die direct in de webinterface aan de onderzoeker wordt gepresenteerd.

11.2. Jupyter-container configuratie

De container fungeert als het laboratorium van de onderzoeker. De inrichting ervan is cruciaal om de technische barrière tussen de gecompileerde .NET-wereld

en de interactieve Python-wereld weg te nemen. Voordat de backend een container kan instantiëren, moet de bijbehorende Docker-image beschikbaar zijn in de lokale image-registry. Deze "jupyter-image" wordt gebouwd met het commando `docker build -t jupyter-image .`, waarbij de Dockerfile uit Bijlage B.5 als blauwdruk dient. Dit build-proces zorgt voor de pre-installatie van alle afhankelijkheden en het inbedden van de verzamelde DLL's, waardoor de opstarttijd van individuele containers minimaal blijft en de reproduceerbaarheid wordt gegarandeerd.

11.2.1. De .NET 10.0 Runtime en Python-omgeving

De Docker-image is opgebouwd rond de `mcr.microsoft.com/dotnet/runtime:10.0` base-image van Microsoft. Door de officiële .NET runtime als basis te nemen, is native ondersteuning voor de C#-DLL's gegarandeerd. Bovenop deze image worden Python 3 en de noodzakelijke pakketten zoals `jupyterlab` en `pythonnet` geïnstalleerd.

11.2.2. Automatische Initialisatie via IPython

Om te voorkomen dat onderzoekers bij het begin van elke notebook een lijst van complexe import-commando's moeten schrijven, wordt gebruikgemaakt van het IPython-configuratiemechanisme. Dit startup-script (zie codefragment B.6 in Bijlage B) zorgt ervoor dat bij elke nieuwe sessie automatisch de CLR wordt geladen en alle relevante namespaces worden geïmporteerd. Hierdoor kan een onderzoeker direct in de notebook types zoals `Rekenfactor` gebruiken, alsof het native Python-klassen zijn.

11.3. Strategie voor Granulaire Implementatie-Switches

Het meest innovatieve aspect van de PoC, en de kern van het plan voor EMAV-web, is de mogelijkheid om op een granulaire manier tussen implementaties te switchen. Dit is niet een keuze voor het hele systeem, maar een keuze die per specifieke berekeningsstap kan worden gemaakt.

11.3.1. Het Switcheable-concept per Module

In de volledige emissierekenkern van EMAV volgen verschillende wiskundige modules elkaar op. De voorgestelde strategie houdt in dat elke module (bijvoorbeeld "Enterische Emissies" of "Mestopslag") wordt behandeld als een uitwisselbaar onderdeel.

Het plan voor de definitieve implementatie in EMAV-web is als volgt:

- **Interface-gedreven ontwerp:** Elke rekenmodule implementeert een gestandaardiseerde interface (bijv. `ICalculator<Tin, Tout>`).
- **UI-gebaseerde configuratie:** In de webinterface van een project kan de onderzoeker per rekenstap een toggle omzetten.

- **Dynamische Factory:** De orchestrator roept voor elke stap een factory aan die controleert of voor deze run de officiële code of de experimentele Python-file gebruikt moet worden.

11.3.2. Realisatie via het Factory-patroon

In de PoC is dit principe gedemonstreerd aan de hand van de CustomerListTransformerFactory. De factory fungeert als de beslissingslaag die de orchestrator ontzorgt.

```
1 public class CustomerListTransformerFactory()  
2 {  
3     public static ICustomerListTransformer  
4     GetCustomerListTransformer(bool python)  
5     {  
6         return python ? new PyCustomerListTransformer() : new  
7         CsCustomerListTransformer();  
8     }  
9 }
```

Codefragment 11.1: Factory voor het dynamisch wisselen tussen C# en Python implementaties.

11.3.3. De Python-Interop in Actie

Indien voor Python wordt gekozen, wordt een PyCustomerListTransformer geïnstantieerd. Deze klasse fungeert als een brug: zij opent de Python-engine binnen de C#-procescontext, beheert de Global Interpreter Lock (GIL) om concurrency-fouten te voorkomen, en laadt het specifieke scriptbestand in.

De onderzoeker kan vervolgens een script aanleveren (bijv. correctie_emissie.py) dat direct door de professionele C#-omgeving wordt uitgevoerd:

11.4. Veiligheid en Validatie

Het toelaten van externe scripts vereist strikte veiligheidsmaatregelen. In de PoC zijn hiervoor de volgende mechanismen geïmplementeerd:

- **Proces-Isolatie:** De containers draaien onder een niet-root gebruiker en hebben strikte CPU- en geheugenlimieten.
- **Data-Isolatie:** Experimenten werken op tijdelijke SQLite-kopieën, waardoor de centrale database nooit wordt vervuild.
- **In-Notebook Validatie:** De hybride opzet laat toe om resultaten van de C#-code en de Python-code direct naast elkaar te vergelijken in grafieken.

```

1 public List<Customer> TransformList(List<Customer> lijst)
2 {
3     lock (PythonLock)
4     {
5         using (Py.GIL())
6         {
7             dynamic sys = Py.Import("sys");
8             sys.path.append(pythonPath);
9             dynamic module = Py.Import("CustomerListTransformer");
10            var result = module.transform(lijst);
11            return result;
12        }
13    }
14 }

```

Codefragment 11.2: C#-methode die via Python.NET een extern script aanroept.

```

1 def transform(customers):
2     for c in customers:
3         # Onderzoeker kan hier eigen experimentele logica toevoegen
4         c.CustomerName = c.CustomerName.upper() + " (Validated)"
5     return customers

```

Codefragment 11.3: Voorbeeld van een Python-script voor experimentele modelaanpassingen.

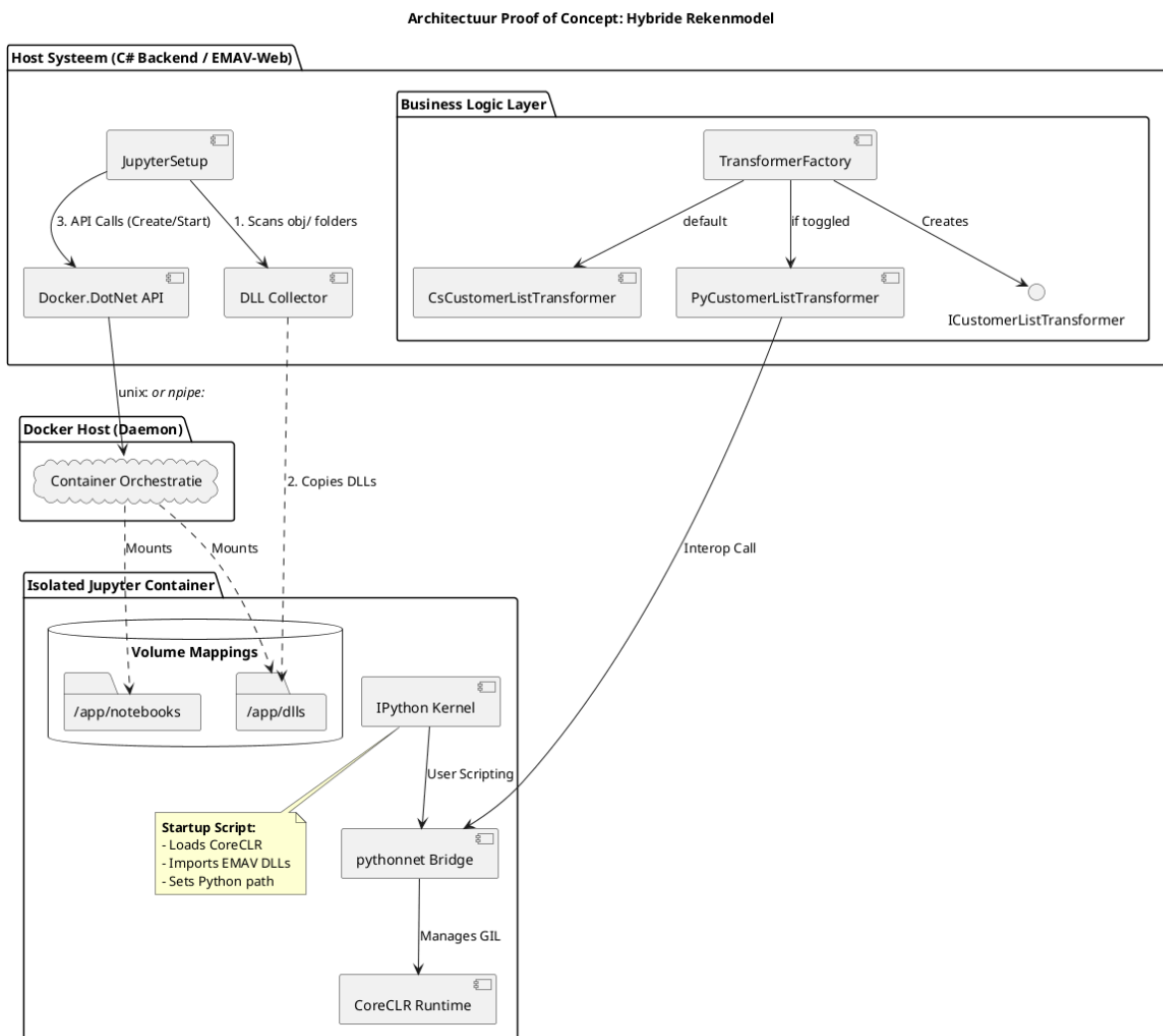
11.5. Resultaten en Beperkingen

Nota betreffende gegevensbescherming: Vanwege de strikte vertrouwelijkheid en het databeleid van ILVO is voor de visualisaties en resultaten in dit hoofdstuk gebruikgemaakt van synthetische datasets. Deze data simuleren de functionele werking van de originele rekenmodellen op een representatieve wijze, zonder bedrijfsgevoelige informatie of intellectueel eigendom te onthullen.

De PoC bewijst dat de switchable architectuur technisch haalbaar en performant is. De interop via Python.NET is snel genoeg om complexe domeinobjecten heen-en-weer te sturen zonder merkbare vertraging. De belangrijkste beperking is de huidige ontwikkelingsstatus van EMAV-web, waardoor de logica nog niet structureel kon worden ingebed in de echte emissierekenmodules.

11.6. Conclusie van de PoC

De ontwikkelde proefopstelling vormt een geslaagde validatie van de hybride strategie. Het toont aan dat de combinatie van .NET 10.0, Docker en Python.NET de



Figuur 11.1: Architecturaal overzicht van de Proof of Concept met C#-orchestratie en hybride Python-integratie.

autonomie van de onderzoekers radicaal kan verhogen zonder in te boeten op de softwarekwaliteit van de backend.

12

Conclusie

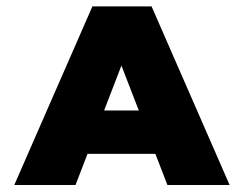
Dit onderzoek richtte zich op de fundamentele vraag hoe de kloof tussen IT-logica en wetenschappelijke analyse binnen de context van ILVO verkleind kan worden. Door de bevindingen en afwegingen uit de opeenvolgende hoofdstukken te synthetiseren, kan een integraal antwoord op de onderzoeksvragen worden geformuleerd.

De uitleg in de *huidige staat van documentatie* leerde dat eerdere pogingen tot documentatie, zoals het bijhouden van “schaduw-code” in losse Markdown-bestanden, faalden door documentation drift. Hoewel deze aanpak voor developers overzichtelijker was dan de lijvige Word-documenten uit het legacy-tijdperk, bleef de informatie gefragmenteerd. De analyse van de *projectcomplexiteit* bevestigde dat een ad-hoc programmeerstijl ontoereikend is voor de hybride datastructuur en legacy-rekenmodules van EMAV. Hoewel alternatieve, eenvoudigere architecturen overwogen zijn, bleek de keuze voor Clean Architecture en een uitgebreid frame van design patterns (zoals Repositories en Strategieën) de enige manier om de wetenschappelijke kern te isoleren. In het hoofdstuk over de *tekortschietsing van documentatie* werd vastgesteld dat zelfs moderne tools zoals DocFX en Swagger, ondanks hun technische kracht, fundamenteel tekortschieten voor onderzoekers omdat zij de intentie en de wiskundige onderbouwing van de modellen niet ontsluiten.

Bij de verkenning van *alternatieve documentatievormen* zijn diverse platforms kritisch tegen elkaar afgewogen. Quarto werd overwogen vanwege het superieure versiebeheer via platte tekst, en Org-mode vanwege de pure implementatie van Literate Programming. Echter, de noodzaak voor een drempelloze interface leidde tot de keuze voor Jupyter-notebooks. Voor de technische koppeling werd Python.NET verkozen boven IronPython; waar IronPython beperkt blijft tot verouderde Python-versies, biedt Python.NET de toegang tot het moderne data-science ecosysteem dat voor onderzoekers essentieel is.

De uitwerking van de *architectuur* en de *Proof-of-Concept* valideerden deze keuzes. Waar directe toegang tot productiedata of het handmatig opzetten van lokale omgevingen te risicovol of complex werd bevonden, boden Docker-sandboxen en lokale data-clonering een veilig alternatief. Het onderzoek bewijst dat een samenwerking van deze technologieën een *Single Source of Truth* realiseert die technisch robuust is en de autonomie van de wetenschapper herstelt.

Concluderend toont dit onderzoek aan dat de hybride integratie van .NET en Jupyter de meest levensvatbare strategie is om de “black-box” van wetenschappelijke software te openen. De belangrijkste beperking is dat een definitieve validatie nog afhankelijk is van een structurele inname van de mening en ervaringen van de onderzoekers zelf. De technische fundering is hiermee gelegd; de menselijke adoptie en verdere automatisatie binnen CI/CD-pipelines vormen de logische vervolgstappen voor de toekomst van EMAV.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Wetenschappelijke instituten zoals ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) maken intensief gebruik van complexe rekenmodellen om emissies, productie-impact en andere landbouwparameters te analyseren. Deze modellen bestaan uit clusters van formules die door onderzoekers worden ontwikkeld en vervolgens door de IT-afdeling worden geïmplementeerd in softwaretoepassingen. Aangezien onderzoekers deze toepassingen gebruiken om hypothesen te testen en scenario's te simuleren, is de kwaliteit, transparantie en aanpasbaarheid van deze implementaties van groot belang. In de praktijk blijken dergelijke modellen echter moeilijk wijzigbaar en weinig inzichtelijk zodra ze in code zijn gegoten.

Binnen ILVO wordt dit probleem concreet zichtbaar in .NET-gebaseerde applicaties waarin emissieformules en bedrijfsparameters zijn verwerkt in C#-logica. Onderzoekers beschikken over diepgaande inhoudelijke expertise om hun modellen en aannames te verfijnen, maar hebben niet altijd voldoende programmeerkennis om deze wijzigingen zelfstandig in de software door te voeren. Voor elke aanpassing zijn zij afhankelijk van de IT-afdeling, waardoor zelfs beperkte experimenten of kleine parameterwijzigingen vertraging oplopen. Bovendien is de bestaande documentatie niet altijd nauw verweven met de code, wat het moeilijk maakt om te achterhalen waar specifieke berekeningen plaatsvinden en welke aannames eraan ten grondslag liggen.

De doelgroep van dit onderzoek bestaat uit twee nauw samenwerkende groepen binnen ILVO: enerzijds de onderzoekers die de wetenschappelijke modellen ont-

wikkelen, en anderzijds de IT-professionals die deze modellen implementeren, onderhouden en documenteren. Beide groepen ondervinden hinder van de huidige werkwijze: onderzoekers ervaren een gebrek aan autonomie en inzicht, terwijl IT-professionals disproportioneel veel tijd besteden aan kleine aanpassingen, ondersteuning en foutopsporing.

Vanuit deze concrete probleemsituatie wordt in deze bachelorproef de volgende centrale onderzoeksvraag geformuleerd: *Hoe kunnen we binnen een .NET-gebaseerde (C# & Razor Pages) onderzoeksapplicatie de kloof verkleinen tussen IT-technische logica en wetenschappelijke analyse, zodat onderzoekers van ILVO zonder diepgaande programmeerkennis inzicht krijgen in de werking van hun intern model en er zelf mee kunnen experimenteren?*

Om deze onderzoeksvraag te kunnen beantwoorden, wordt het onderzoek opgesplitst in een aantal samenhangende deelvragen die zowel het probleemdomain als het oplossingsdomain bestrijken. Eerst wordt onderzocht hoe de huidige samenwerking en workflow tussen onderzoekers en IT-professionals verloopt en waar deze in de praktijk vastloopt. Daarnaast wordt nagegaan welke onderdelen van het bestaande .NET-model en de bijhorende code moeilijk te begrijpen of aan te passen zijn voor onderzoekers. Ook wordt onderzocht in welke mate de huidige documentatie tekortschiet in het zichtbaar maken van berekeningen, aannames en afhankelijkheden binnen het model.

Op basis van deze probleemanalyse richt het onderzoek zich vervolgens op mogelijke oplossingsrichtingen. Daarbij wordt onderzocht welke documentatie- en interactievormen onderzoekers ondersteunen bij het begrijpen van wetenschappelijke modellen, zoals inline annotaties, interactieve documentatie of computationele notatebooks. Verder wordt nagegaan hoe onderzoekers veilig kunnen experimenteren met modelparameters en formules, bijvoorbeeld via simulaties of gevoeligheidsanalyses, zonder de onderliggende softwarestructuur te doorbreken. Tot slot wordt onderzocht hoe documentatie en code nauwer met elkaar verweven kunnen worden zodat wijzigingen aan het model automatisch transparant blijven voor alle betrokkenen.

Het doel van dit onderzoek is het ontwikkelen van een proof-of-concept waarin documentatie, modelberekeningen en experimenteeruimte voor onderzoekers dicht bij elkaar worden gebracht. Dit proof-of-concept moet aantonen hoe onderzoekers op een gecontroleerde en begrijpelijke manier met hun modellen kunnen werken, terwijl de onderhoudbaarheid en technische kwaliteit van de software behouden blijft. Het uiteindelijke resultaat bestaat uit een concreet voorstel en prototype dat ILVO kan ondersteunen bij het toegankelijker, transparanter en duurzamer maken van hun onderzoeksmodellen.

Dit onderzoek wordt bewust afgebakend: het richt zich niet op het herontwerpen van het volledige ILVO-model of het bouwen van een productieklare applicatie, maar op het ontwikkelen en evalueren van technieken, workflows en tooling die

de interactie tussen code en onderzoek verbeteren. Het theoretisch kader wordt gezocht binnen software engineering, documentatieonderzoek, literate programming, domain-driven design en transparantie van wetenschappelijke modellen.

A.2. Literatuurstudie

A.2.1. Literate programming

Literate programming (Knuth, [g.d.](#)) werd geïntroduceerd door Knuth (1992) als een methode waarbij code en documentatie 1 geïntegreerd geheel vormen, eerder dan gescheiden artefacten. De bedoeling is dat er 1 bestand is met de code en de documentatie errond. De tools halen de code eruit en creëren een uitvoerbaar bestand en een ander bestand met de documentatie, dat kan een html web pagina zijn, of een $\text{\LaTeX}$ bestand / pdf. Het kan ook een Markdown bestand zijn.

In plaats van 'code with comments' vertrekt literate programming vanuit het idee dat een programma in de eerste plaats een verhaal is dat aan een menselijke lezer moet worden uitgelegd, waarbij uitvoerbare code een secundaire afgeleide vorm is (Knuth, 1992). Sindsdien zijn er talrijke systemen ontstaan die Knuths principes toepassen in een moderne ontwikkelingsomgeving.

Hoewel Knuths oorspronkelijke WEB en CWEB nog weinig gebruikt worden in de hedendaagse softwarepraktijk, leven zijn ideeën voort in computationele notebooks, literate programminglibraries en research pipelines. Tools zoals Jupyter Notebooks, Org-mode Babel en Quarto implementeren een moderne interpretatie van Knuths 'weave' en 'tangle'-proces, waarin broncode zowel als uitvoerbaar script en als leesbare documentatie dient.

De execution haalt de source code eruit en zorgt voor een uitvoerbaar bestand. De Weave operatie zorgt er voor dat de documentatie uit het source bestand wordt gehaald en die in het gewenste formaat zet. De Tangle operatie verspreidt de source code in de respectievelijke bestanden zelf, niet direct uitvoerbaar.

A.2.2. Org-mode Babel

Org mode is een 'major mode' voor GNU Emacs. Org mode kan voor heel veel gebruikt worden: van notities nemen tot todo-lists, computational notebooks en literate programming (Dominik & Schulte, 2025).

In Org Mode met Babel zijn er 'src-blocks':

```

1  * Src-blocks in org-mode
2  #+BEGIN_SRC python :results verbatim
3      nummer: int = 10
4      return nummer
5  #+END_SRC
6
```

```
7 #+RESULTS:  
8 : 10
```

Hierin kan code worden uitgevoerd terwijl er tekst rond staat. Dit is het idee van literate programming. De meeste 'cellen' in org-mode hebben hun eigen omgeving, maar er bestaan manieren om dit te vermijden. Babel verbetert de src-blocks door een paar zaken te voorzien:

- interactieve en programma executie van code blokken
- code blokken die functies met parameters hebben, kunnen andere code blokken accepteren en oproepen, en
- bestanden exporteren voor literate programming

A.2.3. Jupyter notebook

Jupyter notebooks zijn gemaakt voor python. Deze notebooks worden door vele mensen gebruikt en in verschillende omgevingen. Dit kan gaan van een student die python leert en zijn notities er bij wil schrijven tot een volledige data analyse door professionele onderzoekers. Dit is een standaard voor computationeel onderzoek en datascience (Project Jupyter, [2025](#)). Het werkt met verschillende 'cellen': de markdown cellen en de python cellen. De python cellen verbinden met een virtuele omgeving en voeren hierop telkens hun code uit. Aangezien ze werken op dezelfde omgeving zijn de cellen verbonden aan elkaar. Het grootste verschil met org-mode is dat een .org bestand een plain tekst bestand is. Het is mogelijk om dit bestand te bekijken in eender welke editor. Jupyter notebooks niet. Je hebt een editor nodig die jupyter notebooks ondersteunt. Het bekendste is VsCode van Microsoft. Er bestaat ook de officiële command en app van jupyter zelf die een server opstart en dan in de browser beschikbaar is.

Maar jupyter notebooks hebben zeker hun minpunten. Rule e.a. ([2018](#)) tonen aan dat Jupyter Notebooks niet-lineaire uitvoer geschiedenis hebben, wat reproduceerbaarheid ernstig bemoeilijkt. Ook omdat de cellen in een willekeurige volgorde kunnen worden uitgevoerd, moet er opgelet worden met de run volgorde. De notebooks hebben ook een onzichtbare global state. Het zijn json bestanden, dus versie controle beheer kan hier ook moeilijk mee doen. Aangezien dat alles ook in 1 bestand zit, kan het snel 'bloated' zijn.

A.2.4. RMarkdown

RMarkdown (RStudioPBC, [2020](#)) is ook een computational notebook zoals Jupyter Notebooks, maar het ondersteunt meerdere talen. Het volgt een structuur die sterk aansluit bij de principes van literate programming. Een .Rmd-bestand wordt verwerkt door knitr (Xie e.a., [2025](#)), een transparante engine voor het genereren van

dynamische rapporten in R, ontwikkeld binnen de softwareomgeving voor statistische computing R (The R Foundation, [g.d.](#)).

Knitr voert alle code blokken uit en maakt een nieuw Markdown bestand aan met alle code en hun uitkomsten. Het gemaakte Markdown bestand wordt dan gevalueerd door pandoc (MacFarlane, [2025](#)), die dan het finale product maakt. Vb:

```
1 ---
2 title: "viridis demo"
3 output: html_document
4 ---
5
6 ```{r include - FALSE}
7 library(viridis)
8 ```
9 The code below ...
10
11 ## Colors
12 ```{r}
13 image(volcano, col = viridis(200))
14 ```
```

A.2.5. Tussenbesluit

A.3. Methodologie

Binnen een concrete casus bij het Instituut voor Landbouw-, Visserij- en Voedingsonderzoek (ILVO) wordt een toegepast, onderwerpgericht onderzoeksoptzet gevolgd. De focus ligt op het analyseren van de huidige samenwerking tussen onderzoekers en softwareontwikkelaars rond een .NET-gebaseerd rekenmodel, en op het uitwerken van een haalbare oplossingsrichting die deze samenwerking ondersteunt. De methodologie is opgebouwd uit vijf opeenvolgende fasen, waarin zowel het probleemdomein als het oplossingsdomein systematisch worden onderzocht.

A.3.1. Eerste fase

In de eerste fase wordt het probleemdomein in kaart gebracht. Hierbij wordt onderzocht hoe onderzoekers en ontwikkelaars momenteel samenwerken rond het bestaande model, waar verantwoordelijkheden liggen en op welke punten de workflows elkaar belemmeren. Deze analyse vertrekt vanuit concrete use-cases, zoals het aanpassen van emissieformules, het testen van alternatieve parameters en het uitvoeren van gevoeligheidsanalyses. De gegevens worden verzameld via documentanalyse en overlegmomenten met betrokkenen, met als doel inzicht krijgen

in waar de kloof tussen wetenschappelijke logica en technische implementatie zich manifesteert.

A.3.2. Tweede fase

De tweede fase richt zich op het identificeren van noden en vereisten van beide doelgroepen. Voor onderzoekers gaat het onder meer om transparantie van berekeningen, experimenteerruimte en begrijpelijke documentatie; voor ontwikkelaars om onderhoudbaarheid, stabiliteit en controle over data en interfaces. Deze noden worden gestructureerd geformuleerd als functionele en niet-functionele requirements, die richtinggevend zijn voor het verdere onderzoek. Hierbij wordt expliciet rekening gehouden met organisatorische aspecten, zoals taakverdeling en communicatiestromen, in lijn met inzichten uit onder meer Conway's Law (Conway, 1968).

A.3.3. Derde fase

In de derde fase wordt het oplossingsdomein onderzocht aan de hand van een gerichte literatuurstudie en analyse van bestaande tools en workflows. Concepten zoals Knuth (g.d.), computational notebooks (bijvoorbeeld jupyter notebook (Project Jupyter, 2025) en quarto (Posit Software, PBC, g.d.)), documentatieplatformen (zoals docFX (.NET Foundation, 2025)) en model life-cycle tools (zoals mlflow (LF AI & Data Foundation, 2025)) worden bestudeerd op hun relevantie voor gelijkaardige samenwerkingsproblemen. Daarbij wordt nagegaan in welke mate deze benaderingen geschikt zijn binnen een .NET-context, en welke principes overdraagbaar zijn, ook wanneer de tools zelf technologisch niet rechtstreeks inzetbaar zijn.

A.3.4. Vierde fase

Op basis van de voorgaande fasen volgt in de vierde fase een selectie en ontwerp van een oplossingsrichting. De eerder geïdentificeerde requirements worden gebruikt om mogelijke oplossingen af te wegen en te beperken tot een haalbare subset. Hierbij wordt bewust gekozen voor een beperkte scope, met focus op het verbeteren van transparantie, documentatie en experimenteermogelijkheden, zonder het bestaande model volledig te herontwerpen. Het resultaat van deze fase is een concreet concept voor een ondersteunende workflow of tool, afgestemd op de ILVO-casus.

A.3.5. Vijfde fase

In de vijfde en laatste fase wordt deze oplossingsrichting geconcretiseerd in een proof-of-concept. Dit prototype demonstreert hoe onderzoekers inzicht kunnen krijgen in modelberekeningen en parameters, en hoe zij op een gecontroleerde manier kunnen experimenteren zonder diepgaande programmeerkennis. Het proof-of-concept wordt geevalueerd aan de hand van de eerder gedefinieerde

use-cases en requirements. De bevindingen uit deze evaluatie vormen de basis voor conclusies en aanbevelingen over toepasbaarheid en meerwaarde van de voorgestelde aanpak binnen ILVO en gelijkaardige onderzoekscontexten.

A.4. Verwacht resultaat, conclusie

A.4.1. Probleem- en domeinanalyse

Een helder overzicht van de *current flow* (onderzoekers → IT → applicatie)

Een lijst van problemen, zoals:

- beperkte transparantie
- onvoldoende verweven documentatie
- moeilijk traceerbaarheid van berekeningen
- lange feedbackloops

A.4.2. Literatuurstudie & technologieverkenning

Een lijst van mogelijke technieken en tools om de kloof tussen code en onderzoek te verkleinen. Ook bepalen welke technieken *haalbaar* en *relevant* zijn voor ILVO.

A.4.3. Proof-of-Concept ontwikkeling

Een geïntegreerde POC die aantoont hoe onderzoekers:

- parameters kunnen wijzigen,
- resultaten kunnen testen,

zonder afhankelijk te zijn van IT voor kleine experimenten.

A.4.4. Evaluatie en validatie

Conclusie over welke aanpak het meest effectief is om de kloof te verkleinen en een concreet advies voor ILVO met mogelijke roadmap.

B

Codefragmenten

B.1. C# Implementatie details

B.2. Container Configuratie

```
1  import clr
2  clr.AddReference("System.Windows.Forms")
3  clr.AddReference("System.Drawing")
4  clr.AddReference("System")
5  clr.AddReference("System.Windows")
6  from pythonnet import load
7  from System import *
8  from System.Drawing import Color, Point, Size
9  from System.Windows.Forms import Application, Button, Form,
    MessageBox, TextBox
10 from System.Windows import Clipboard, DataFormats, Window
11 from System.Windows.Threading import Dispatcher
12 class MainForm(Form):
13     def __init__(self):
14         super().__init__()
15         self.Text = ".NET in python"
16         self.Size = Size(400, 600)
17 def simple_form():
18     global _application_initialized
19
20     try:
21         if not _application_initialized:
22             Application.EnableVisualStyles()
23             Application.SetCompatibleTextRenderingDefault(False)
24             _application_initialized = True
25
26         Application.Run(MainForm())
27 except Exception as e:
28     print(f"Error in simple_form: {e}")
29     return None
```

Codefragment B.1: Codefragment gemaakt als voorbeeld

```
1 private static void SetupDll()  
2 {  
3     var assemblyLocation = Path.GetDirectoryName(  
4         Assembly.GetExecutingAssembly().Location!);  
5     var solutionRoot = Path.GetFullPath(  
6         Path.Combine(assemblyLocation, "..", "..", "..", "..")  
7     );  
8  
9     var targetDir = Path.Combine(solutionRoot, "ILVO.EMAV.Jupyter",  
10        "dlls");  
11    Directory.CreateDirectory(targetDir);  
12  
13    var dllFiles = Directory.GetFiles(solutionRoot, "*.dll",  
14        SearchOption.AllDirectories)  
15        .Where(path => path.Contains(@"\obj\"))  
16        .Where(path => !path.Contains(@"\ref\"))  
17        .Where(path => !path.Contains(@"\runtimes\"));  
18  
19    foreach (var dll in dllFiles)  
20    {  
21        var fileName = Path.GetFileName(dll);  
22        var destPath = Path.Combine(targetDir, fileName);  
23        File.Copy(dll, destPath, overwrite: true);  
24        Console.WriteLine($"Copied: {fileName}");  
25    }  
26 }
```

Codefragment B.2: Implementatie van de SetupDll-methode voor het verzamelen van DLL-bestanden.

```
1 var response = await docker.Containers.CreateContainerAsync(
2     new CreateContainerParameters
3     {
4         Image = "jupyter-image",
5         ExposedPorts = new Dictionary<string, EmptyStruct>
6         {
7             { "8888/tcp", default }
8         },
9         HostConfig = new HostConfig
10        {
11            PortBindings = new Dictionary<string, IList<PortBinding>>
12            {
13                {
14                    "8888/tcp", new List<PortBinding>
15                    {
16                        new PortBinding { HostPort = "" }
17                    }
18                }
19            }
20        }
21    }
22 );
23
24 await docker.Containers.StartContainerAsync(response.ID, null);
```

Codefragment B.3: Aanmaken en opstarten van de Jupyter Docker-container via Docker.DotNet.

```

1  var logs = await
    docker.Containers.GetContainerLogsAsync(response.ID, true,
2      new ContainerLogsParameters
3      {
4          ShowStderr = true,
5          ShowStdout = true,
6          Follow = false
7      });
8
9  var output = await DemultiplexStream(logs);
10
11  var tokenMatch = Regex.Match(output, @"token=([a-f0-9]+)");
12  var token = tokenMatch.Success ? tokenMatch.Groups[1].Value : "";
13
14  var portMatch = Regex.Match(output, @"http://[^:]+:(\d+)");
15  var port = portMatch.Success ? portMatch.Groups[1].Value : "8888";
16
17  var jupyterUrl = $"http://localhost:{port}/?token={token}";

```

Codefragment B.4: Extractie van de Jupyter-token en poort uit de container-logs.

```

1  FROM mcr.microsoft.com/dotnet/runtime:10.0
2
3  RUN apt-get update && apt-get install -y python3 python3-pip
4
5  RUN pip3 install jupyterlab pythonnet ipykernel
    --break-system-packages
6
7  WORKDIR /app
8
9  COPY dlls /app/dlls
10  COPY notebooks /app/notebooks
11  COPY startup.py /root/.ipython/profile_default/ipython_config.py
12
13  EXPOSE 8888
14
15  CMD ["jupyter", "lab", "--ip=0.0.0.0", "--allow-root",
    "--no-browser", \
16  "--notebook-dir=/app/notebooks"]

```

Codefragment B.5: Dockerfile voor de Jupyter-omgeving met .NET-ondersteuning.

```
1 c = get_config()
2 c.InteractiveShellApp.exec_lines = [
3     'import sys',
4     'import pythonnet',
5     'pythonnet.load("coreclr")',
6     'import clr',
7     'sys.path.append("/app/dlls")',
8     'clr.AddReference("ILVO.EMAV.Core")',
9     'clr.AddReference("ILVO.EMAV.Data.Web")',
10    'clr.AddReference("ILVO.EMAV.Infrastructure.Exporter")',
11    # ... verdere references
12    'from ILVO.EMAV.Core import *',
13    'from ILVO.EMAV.Data.Rekenfactoren import *',
14    # ... verdere imports
15 ]
```

Codefragment B.6: Startup-script voor IPython voor het automatisch inladen van .NET-assemblies.

Bibliografie

- Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79. <https://doi.org/10.1145/2723872.2723882>
- Bogard, J. (2018). *Vertical Slice Architecture* [geraadpleegd op 2026-05-27]. <https://jimmybogard.com/vertical-slice-architecture/>
- Bosch, J., Olsson, H. H. & Crnkovic, I. (2021). Towards MLOps: A Framework and Maturity Model. *Proceedings of the 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 1–8. <https://doi.org/10.1109/SEAA53835.2021.00010>
- Bui, T. (2015). Analysis of Docker Security. https://doi.org/https://www.researchgate.net/publication/270906436_Analysis_of_Docker_Security
- Comella-Dorda, S., Wallnau, K. C., Seacord, R. C. & Robert, J. E. (2000). *A Survey of Legacy System Modernization Approaches* (tech. rap. CMU/SEI-2000-TN-003) [Geraadpleegd op 2026-05-27]. Carnegie Mellon University, Software Engineering Institute. <https://www.sei.cmu.edu/library/a-survey-of-legacy-system-modernization-approaches/>
- contributors of the Python.NET project. (2026). *pythonnet - Python.NET* [[Source Code], Geraadpleegd op 2026-02-25]. <https://github.com/pythonnet/pythonnet>
- Conway, M. (1968). Committees paper [Geraadpleegd op 20-01-2026]. *Datamation magazine*.
- Docker Inc. (2026a). *docker container run* [Geraadpleegd op 2026-05-27]. <https://docs.docker.com/reference/cli/docker/container/run/>
- Docker Inc. (2026b). *Docker Desktop architecture* [Geraadpleegd op 2026-05-27]. <https://docs.docker.com/desktop/architecture/>
- Docker Inc. (2026c). *Docker Engine Security* [Geraadpleegd op 2026-05-27]. <https://docs.docker.com/engine/security/>
- Docker Inc. (2026d). *Publishing and Exposing Ports* [Geraadpleegd op 2026-05-27]. <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
- Docker Inc. (2026e). *Resource Constraints* [Geraadpleegd op 2026-05-27]. https://docs.docker.com/engine/containers/resource_constraints/

- Dominik, C. & Schulte, E. (2025, oktober 24). *Org mode for Gnu Emacs* [geraadpleegd op 2025-12-10]. Verkregen 24 oktober 2025, van <https://orgmode.org/index.html>
- Feathers, M. C. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Fowler, M. (2003). *Repository*. <https://martinfowler.com/eaCatalog/repository.html>
- Fowler, M. (2011). *CQRS*. <https://martinfowler.com/bliki/CQRS.html>
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- Ibrahim, W. M., Bettenburg, N., Adams, B. & Hassan, A. E. (2012). On the Relationship Between Comment Update Practices and Software Bugs. *Journal of Systems and Software*, 85(10), 2293–2304. <https://doi.org/10.1016/j.jss.2011.09.019>
- IPython Development Team. (g.d.). Overview of the IPython configuration system: Startup Files [Geraadpleegd op 2026-05-27]. <https://ipython.readthedocs.io/en/2.x/development/config.html#startup-files>
- Johnson, A. & Fotouhi, F. (1996). The SANDBOX: Scientists Accessing Necessary Data Based On eXperimentation. *Proceedings of the Seventh International Working Conference on Scientific and Statistical Database Management*, 130–139. <https://doi.org/10.1109/SSDM.1994.336440>
- Jupyter, P., Bussonnier, M., Forde, J., Freeman, J., Granger, B., Head, T., Holdgraf, C., Kelley, K., Nalvarte, G., Osheroff, A., Pacer, M., Panda, Y., Perez, F., Ragan-Kelley, B. & Willing, C. (2018). Binder 2.0 - Reproducible, interactive, sharable environments for science at scale. In F. Akici, D. Lippa, D. Niederhut & M. Pacer (Red.), *Proceedings of the 17th Python in Science Conference* (pp. 113–120). <https://doi.org/10.25080/Majora-4aflf417-011>
- Khononov, V. (2022). *Learning Domain-Driven Design*. O'Reilly Media.
- Khorikov, V. (2020). *Functional C#: Handling failures and input errors* [geraadpleegd op 2026-05-27]. <https://enterprisecraftsmanship.com/posts/functional-c-handling-failures-input-errors/>
- Knuth, D. E. (g.d.). *Literate Programming* [Accessed 10 December 2025]. Verkregen 10 december 2025, van <http://www.literateprogramming.com/>
- Knuth, D. E. (1992). *Literate Programming*. Center for the Study of Language; Information.
- Kohavi, R., Tang, D. & Xu, Y. (2020). *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press. <https://doi.org/10.1017/9781108653985>
- LF AI & Data Foundation. (2025). *MLflow: An open source platform for the Machine Learning Lifecycle* [[Open-source software], Geraadpleegd op 2026-01-21]. <https://mlflow.org/>

- LiteBus Contributors. (2024). *LiteBus: A lightweight message bus implementation for .NET*. <https://github.com/LiteBus/LiteBus>
- MacFarlane, J. (2025). *Pandoc, The universal document converter*. <https://pandoc.org/>
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Meng, M., Steinhardt, S. & Schubert, A. (2018). Application Programming Interface Documentation: What Do Software Developers Want? *Journal of Technical Writing and Communication*, 48(3), 295–330. <https://doi.org/10.1177/0047281617721853>
- Microsoft. (2026). *Common Language Runtime* [[Documentation], Geraadpleegd op 2026-02-28]. <https://docs.microsoft.com/en-us/dotnet/standard/clr>
- Millett, S. & Tune, N. (2015). *Patterns, Principles, and Practices of Domain-Driven Design*. Wrox.
- Mono Project. (2026). *Home / Mono* [[Source Code], Geraadpleegd op 2026-03-01]. <https://www.mono-project.com>
- .NET Foundation. (2025). *DocFX: Documentation Generator for .NET* [[Source Code], Geraadpleegd op 2026-01-21]. <https://github.com/dotnet/docfx>
- .NET foundation. (2026). *IronPython* [[Source Code], Geraadpleegd op 2026-03-01]. <https://ironpython.net>
- Posit Software, PBC. (g.d.). *Official quarto github repository* [geraadpleegd op 2025-12-10]. <https://github.com/quarto-dev/quarto-cli/>
- Project Jupyter. (2025). *Project Jupyter / Home* [geraadpleegd op 2025-12-11]. <https://jupyter.org/>
- Python Software Foundation. (2026). *GitHub - python/cpython: The Python programming language* [[Source Code], Geraadpleegd op 2026-02-28]. <https://github.com/python/cpython>
- Richardson, C. (2018). *Microservices Patterns: With examples in Java* [Geraadpleegd op 2026-05-27]. Manning Publications.
- RStudioPBC. (2020). *R Markdown* [geraadpleegd op 2025-12-12]. <https://rmarkdown.rstudio.com/>
- Rule, A., Tabard, J. D. & Hollan, J. D. (2018). Exploration and Explanation in Computational Notebooks [geraadpleegd op 2025-12-11]. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 32, 1–12. <https://doi.org/10.1145/3173574.3173606>
- Seacord, R. C., Comella-Dorda, S., Lewis, G., Place, P. R. & Plakosh, D. (2001). *Legacy System Modernization Strategies* (tech. rap. CMU/SEI-2001-TR-025) [Geraadpleegd op 2026-05-27]. Carnegie Mellon University, Software Engineering In-

- stitute. <https://www.sei.cmu.edu/library/legacy-system-modernization-strategies/>
- Seemann, M. & van Deursen, S. (2019). *Dependency Injection: Principles, Practices, and Patterns*. Manning Publications.
- SmartBear Software. (2026). *Paths and Operations* [Geraadpleegd op 2026-05-27]. https://swagger.io/docs/specification/v3_0/paths-and-operations/
- Tamburri, D. A. (2020). Sustainable MLOps: A Research Roadmap. *IEEE Software*. <https://doi.org/10.1109/SYNASC51798.2020.00015>
- Tan, W. S., Wagner, M. & Treude, C. (2024). Detecting Outdated Code Element References in Software Repository Documentation. *Empirical Software Engineering*, 29(5). <https://doi.org/10.1007/s10664-023-10397-6>
- The R Foundation. (g.d.). *R: The Project for Statistical Computing* [Geraadpleegd op 2025-12-12]. <https://www.r-project.org/>
- Xie, Y. e.a. (2025). *knitr* (Versie 1.50) [Source code. Geraadpleegd op 12-10-2025]. <https://github.com/yihui/knitr>
- Zheng, L. & Wenhao, J. (2020). Vulnerability Analysis and Security Research of Docker Container. *IEEE Access*, 8, 6573–6586. <https://ieeexplore.ieee.org/document/9236837>